

Раздел 3. ГЛОБАЛЬНАЯ СЕТЬ *Internet*

Слайд 3

3.1. Введение в *Internet*

Глобальная сеть Internet реализована на основе стека сетевых протоколов TCP/IP, обеспечивающих передачу данных между разнородными локальными и территориальными сетями (подсетями), а также коммуникационными системами и устройствами.

Краткая история

Появлению сети Internet и стека протоколов TCP/IP предшествовала в середине 1960-х годов разработка под эгидой агентства DARPA (Defence Advanced Research Projects Agency – Управление перспективных исследований Министерства обороны США) сети, получившей название ARPANET (Advanced Research Projects Agency NETwork). Разработка сети была поручена Стэнфордскому исследовательскому институту и трём американским университетам: Калифорнийскому в Лос-Анжелесе и Университетам штата Юта и штата Калифорния в Санта-Барбаре. Экспериментальная сеть из четырёх узлов была запущена в конце 1969 года, а к концу 1972 года в сети насчитывалось более 30 узлов.

В 1974 году были разработаны модели и протоколы TCP/IP для управления обменом данными в интерсетях, а 1 января 1983 года сеть ARPANET полностью перешла на протокол TCP/IP.

В конце 1970-х годов Национальный научный фонд США (National Science Foundation, NSF) начал разработку межуниверситетской сети, получившей название NSFNet, которая имела гораздо бóльшую пропускную способность, чем ARPANET. В середине 1980-х годов произошло объединение сетей NSFNet и ARPANET, за которым закрепилось название INTRNET (Интернет).

В 1984 году была разработана система доменных имён (Domain Name System, DNS), а в 1989 году появилась концепция Всемирной паутины (World Wide Web, WWW) и были разработаны протокол передачи гипертекста HTTP (HyperText Transfer Protocol) и язык разметки гипертекста HTML (HyperText Markup Language).

Интернет объединил большинство существующих сетей и к началу 21 века стал популярным средством для обмена данными.

В настоящее время подключиться к Интернету можно через спутники связи, радиоканалы, кабельное телевидение, телефон, сотовую связь, специальные оптоволоконные линии или электропровода.

Специфические особенности глобальной сети Internet

1. Неограниченный территориальный охват.
2. Сеть объединяет подсети (в том числе локальные сети) разных технологий и компьютеры разных классов (от персональных до суперкомпьютеров).

3. Для передачи данных на большие расстояния используется аппаратура передачи данных (модемы, модемные терминалы) и активное сетевое оборудование (маршрутизаторы, коммутаторы).

4. Топология глобальных сетей, в общем случае, произвольная многосвязная.

5. Одна из важнейших задач – организация эффективной маршрутизации передаваемых данных.

6. Глобальная сеть содержит каналы связи разных типов с пропускными способностями до сотен Гбит/с: кабельные оптические и электрические, в том числе телефонные, беспроводные наземные и спутниковые каналы.

Достоинства сети Internet

1. Неограниченный доступ к любым вычислительным и информационным ресурсам, а также множеству специфических услуг (электронная почта, голосовая связь, конференцсвязь, телевидение по запросу и т.д.).

2. Возможность доступа к ресурсам сети практически из любой точки Земного шара.

3. Возможность передачи любых видов данных, в том числе мультимедийных (аудио, видео).

Слайд 4

Архитектурная концепция

Структура сети Internet может быть представлена как множество компьютеров, называемых **хостами**, подключенных к некоторой единой интерсети, представляющей собой совокупность физических сетей, называемых **подсетями**. В качестве подсетей могут выступать *локальные сети*, работающие под управлением некоторых аппаратно зависимых протоколов (Ethernet, Token Ring), или *коммуникационные системы* произвольной физической природы (модемные коммутируемые или выделенные линии, сети X.25, Frame Relay, FDDI, ATM и др.). При этом все функции протокола IP выполняют **хосты** и **маршрутизаторы**, называемые **узлами** сети.

Слайд 5

Стек протоколов TCP/IP

Под **стеком (семейством) протоколов TCP/IP** в широком смысле обычно понимают весь набор реализаций стандартов RFC (Request for Comments).

Основным протоколом стека TCP/IP является **протокол IP**, который обеспечивает:

- *негарантированную доставку* пакетов, т.к. передаваемые по сети пакеты могут быть утеряны, дублированы, задержаны, доставлены с нарушением порядка;

- *дейтаграммную доставку без установления соединения*, то есть каждый пакет представляет собой обрабатываемый независимо от других блок данных, причем последовательно исходящие от отправителя пакеты могут распространяться по различным путям в сети, менять порядок и даже теряться;

- *максимально возможную доставку* пакетов в том смысле, что потеря пакета происходит лишь в той ситуации, когда протокол не находит никаких физических средств для его доставки.

Соответствие уровней TCP/IP уровням OSI-модели и используемые на каждом уровне основные протоколы стека TCP/IP представлены на слайде.

Модель стека протоколов TCP/IP содержит 4 уровня.

На первом уровне (**Network interface – сетевой интерфейс**) находится аппаратно зависимое программное обеспечение, реализующее передачу данных в той или иной среде. Среда передачи данных может быть реализована различными способами: от простого двухточечного звена до сложной многоузловой коммуникационной структуры сети X.25 или Frame Relay. Стек протоколов TCP/IP поддерживает все стандартные протоколы физического и канального уровней различных сетевых технологий: Ethernet, Token Ring, FDDI, PPP и другие.

На втором уровне (**Internet – межсетевой**) реализуется задача маршрутизации с использованием протокола IP. Вторая важная задача протокола IP – сокрытие аппаратно-программных особенностей среды передачи данных и предоставление вышележащим уровням единого унифицированного и аппаратно независимого интерфейса для доставки данных, что обеспечивает многоплатформенное применение приложений, работающих под TCP/IP.

На третьем уровне (**Transport – транспортный**) решаются задачи надежной доставки пакетов и сохранение их порядка и целостности.

На четвертом уровне (**Application – прикладной**) находятся прикладные задачи, запрашивающие сервис у транспортного уровня.

Основными особенностями стека протоколов TCP/IP являются:

- независимость от среды передачи данных;
- негарантированная доставка пакетов.

Информационные объекты (данные), передаваемые на разных уровнях в сети Интернет, получили следующие наименования:

- *сообщение (message)* – блок данных, которым оперирует прикладной уровень, передаваемый от приложения к транспортному уровню с соответствующими этому приложению размером и семантикой;

- *сегмент (segment)* – блок данных, которым оперирует протокол TCP на транспортном уровне;

- *дейтаграмма (datagram)* – блок данных, которым оперирует протокол UDP на транспортном уровне;

- *пакет (packet)* – блок данных, называемый также IP-дейтаграммой, которым оперирует протокол IP на межсетевом уровне;

- *кадр (frame)* – аппаратно зависимые блоки данных, полученные в результате упаковки IP-дейтаграмм в формат, приемлемый для данной

физической среды передачи данных и передаваемый на нижнем уровне TCP/IP-модели, называемом «сетевым интерфейсом». Рассмотрим кратко основные протоколы стека TCP/IP, некоторые из которых (UDP, TCP, IP, ICMP, DHCP, ARP, RIP, OSPF, SLIP, HDLC, PPP) более подробно рассматриваются ниже.

Слайд 6

Протоколы стека TCP/IP прикладного уровня 4

На прикладном уровне реализуются процессы, связанные с выполнением **прикладных программ**, которые можно разбить на 2 группы: **пользовательские**, обеспечивающие решение конкретных пользовательских задач, и **системные**, предоставляющие пользователям множество стандартных услуг, например: обмен файлами, организация электронной почты, контроль и управление сетевыми устройствами и приложениями в сети и т.п. Системные приложения называются **протоколами прикладного уровня** и имеют конкретные названия и соответствующую аббревиатуру.

В зависимости от вида предоставляемых услуг и требований к их реализации прикладные программы в качестве транспорта используют протоколы транспортного уровня: TCP или UDP.

Множество протоколов прикладного уровня можно разбить на классы в зависимости от предоставляемых услуг на протоколы:

- передачи файлов (FTP, TFTP, HTTP);
- почтовые (SMTP, POP3, IMAP);
- контроля и управления сетью (SNMP, DHCP, SIP, DNS);
- прочие (TELNET, BGP, RTP, ...).

Протоколы передачи файлов

FTP (File Transfer Protocol) – протокол передачи файлов, предназначенный для передачи файлов в сети и доступа к удалённым хостам, реализует следующие функции:

- подключение к серверам FTP;
- просмотр содержимого каталогов;
- загрузка файлов с сервера или на сервер.

FTP функционирует поверх транспортного протокола TCP и использует порт 20/TCP для передачи данных и порт 21/ TCP для передачи команд. В протоколе FTP предусмотрены возможности аутентификации и передачи файла с прерванного места, если передача файла была прервана по какой-то причине.

TFTP (Trivial File Transfer Protocol – простой протокол передачи файлов) предназначен главным образом для первоначальной загрузки бездисковых рабочих станций. TFTP использует транспортный протокол UDP и порт 69/UDP. В отличие от FTP, протокол TFTP не содержит возможностей аутентификации, хотя возможна фильтрация по IP-адресу.

HTTP (HyperText Transfer Protocol – протокол передачи гипертекста) предназначен для передачи данных (изначально – в виде гипертекстовых

документов) на основе клиент-серверной технологии. HTTP в настоящее время используется во Всемирной паутине для получения информации с веб-сайтов.

Почтовые протоколы

SMTP (Simple Mail Transfer Protocol) – простой протокол передачи почты, предназначенный для передачи электронной почты в сетях TCP/IP.

POP3 (Post Office Protocol Version 3) – протокол почтового отделения, версия 3, обычно используемый почтовым клиентом в паре с протоколом SMTP для получения сообщений электронной почты с сервера. Протокол POP3 использует транспортный протокол TCP и порт 110/TCP. Альтернативным протоколом для сбора сообщений с почтового сервера является протокол IMAP.

IMAP (Internet Message Access Protocol) – протокол доступа к электронной почте Интернета, как и POP3, служит для работы со входящими письмами, однако обеспечивает ряд дополнительных функций, предоставляя пользователю доступ к хранилищу электронных писем на сервере так, как будто эти письма находятся на его компьютере. POP3 использует транспортный протокол TCP и порт 143/TCP. Для отправки писем используется протокол SMTP.

Протоколы контроля и управления сетью

SNMP (Simple Network Management Protocol – протокол простого управления сетями) предназначен для управления и контроля за сетевыми устройствами и приложениями в сети передачи данных путём обмена управляющей информацией. Протокол SNMP встроен во все сетевые ОС и использует транспортный протокол UDP и порты 161/UDP и 162/UDP.

DHCP (Dynamic Host Configuration Protocol – протокол динамической конфигурации узла) предназначен для автоматического распределения между компьютерами IP-адресов и конфигурационных параметров, необходимых для работы в сети TCP/IP. Протокол реализуется в так называемом DHCP-сервере по клиент-серверной технологии путём выдачи IP-адреса и конфигурационных параметров в ответ на поступивший запрос от компьютера. Протокол DHCP использует транспортный протокол UDP и порты 67/UDP и 68/UDP.

DNS (Domain Name System – система доменных имён) представляет собой компьютерную распределённую иерархическую систему для получения информации о доменах, чаще всего для получения IP-адреса по символному имени хоста (компьютера или устройства). Распределённая база данных DNS поддерживается с помощью иерархии DNS-серверов, взаимодействующих по одноимённому протоколу. Протокол DNS встроен во все сетевые ОС и использует транспортные протоколы TCP и UDP и, соответственно, порты 53/TCP и 53/UDP.

SIP (Session Initiation Protocol) – протокол установления сеанса, предназначенный для установления и завершения пользовательского интернет-сеанса, включающего обмен мультимедийным содержимым (видео- и аудиоконференции, онлайн-игры).

Прочие протоколы

TELNET (TELEtype NETwork) – виртуальный текстовый терминал, предназначенный для реализации текстового интерфейса в сети с использованием транспортного протокола TCP (стандартный порт 23/TCP).

BGP (Border Gateway Protocol – протокол граничного шлюза) – основной протокол динамической маршрутизации в Интернете, предназначенный для обмена информацией о маршрутах между автономными системами. Функционирует поверх протокола транспортного уровня TCP и использует порт 179/TCP.

RTP (Real-time Transport Protocol) – протокол реального времени, предназначенный для передачи трафика реального времени. Заголовок RTP-пакета содержит данные, необходимые для восстановления голоса или видеоизображения в приёмном узле, о типе кодирования информации (JPEG, MPEG и т. п.) а также временную метку и номер пакета. Эти параметры позволяют при минимальных задержках определить порядок и момент декодирования каждого пакета, а также интерполировать потерянные пакеты. В качестве нижележащего протокола транспортного уровня, *как правило, используется протокол UDP.*

Слайд 7

Протоколы стека TCP/IP уровней 1-3

К протоколам стека TCP/IP уровней 1-3 относятся:

- транспортные протоколы уровня 3 (TCP, UDP и др.);
- протоколы межсетевого уровня 2 (IP, ISMP, IGMP, RIP, OSPF, ARP, RARP и др.);
- протоколы сетевого доступа уровня 1 (SLIP, HDLC, PPP).

Протоколы транспортного уровня

TCP (Transmission Control Protocol) – протокол управления передачей данных с установлением соединения, реализующий обмен данными между двумя узлами на основе некоторого соглашения об управлении потоком данных.

UDP (User Datagram Protocol) – дейтаграммный протокол передачи данных в виде независимых единиц – дейтаграмм (datagram).

Протоколы межсетевого уровня

IP (Internet Protocol) - основной протокол стека TCP/IP, реализующий передачу пакетов по IP-сети от узла к узлу без гарантии доставки пакетов, сохранения целостности пакетов и порядка потока пакетов.

ICMP (Internet Control Message Protocol) – межсетевой протокол управляющих сообщений, используемый в основном для передачи сообщений об ошибках и исключительных ситуациях, возникших при передаче данных, а также выполняющий некоторые сервисные функции. ICMP-сообщения генерируются при нахождении ошибок в заголовке IP пакета, при отсутствии

маршрута к адресату, а также используются маршрутизаторами для обновления записей в таблице маршрутизации отправителя и для управления скоростью отправки сообщений отправителем. ICMP-сообщения инкапсулируются в IP пакеты.

RIP (Routing Information Protocol) – протокол маршрутизации типа DVA, реализующий алгоритм обмена информацией о доступных сетях и расстояниях до них путём *периодической* рассылки широковещательных пакетов.

OSPF (Open Shortest Path First) – протокол маршрутизации типа LSA, реализующий алгоритм обмена информацией о состоянии каналов, путём периодического тестирования состояния каналов с соседними маршрутизаторами. Протокол OSPF разработанный для применения в сети Интернет и используется в других больших сетях (NetWare, SNA, XNS, DECNet).

ARP (Address Resolution Protocol – Протокол разрешения адресов) предназначен для определения физического адреса устройства (MAC-адреса) по его IP-адресу.

RARP (Reverse Address Resolution Protocol – Протокол обратного определения адреса) предназначен для определения IP-адреса устройства по его физическому адресу (MAC-адресу).

Протоколы **ARP** и **RARP** часто рассматривают как протоколы промежуточного уровня 2,5 *OSI-модели*.

IGMP (Internet Group Management Protocol) – протокол управления группами Интернета, предназначенный для управления групповой (multicast) передачей данных в IP-сетях. IGMP используется маршрутизаторами и IP-узлами для организации групп сетевых устройств, а также для поддержки потокового видео и онлайн-игр, обеспечивая эффективное использование сетевых ресурсов.

Протоколы канального уровня («сетевой интерфейс»)

SLIP (Serial Line IP) – первый стандарт канального уровня для выделенных линий, разработанный специально для стека протоколов TCP/IP, который благодаря простоте может использоваться как для коммутируемых, так и для выделенных каналов. SLIP поддерживается только протоколом сетевого уровня IP.

HDLC (High-level Data Link Control Procedure) – высокоуровневый протокол управления каналом – стандарт ISO для выделенных линий, представляющий собой семейство протоколов LAP (Link Access Protocol). HDLC относится к бит-ориентированным протоколам.

PPP (Point-to-Point Protocol) – протокол двухточечного соединения, пришедший на смену протоколу SLIP и построенный на основе формата кадров протоколов семейства HDLC с дополнением собственных полей. PPP является стандартным протоколом Интернета и так же, как протокол HDLC, представляет собой семейство протоколов.

Слайд 8

Концептуальная модель передачи данных в сетях TCP/IP

На слайде иллюстрируется концептуальная модель передачи данных в сетях TCP/IP.

В хостах реализуются протоколы всех четырех уровней модели стека TCP/IP, а в маршрутизаторах – второго и третьего уровней.

Слайд 9

Виды трафика в сети Internet

Поток передаваемых данных (трафик) в компьютерных сетях делится на четыре вида:

- **unicast** (индивидуальный) – поток данных, предназначенных только одному адресату;
- **multicast** (групповой) – поток данных, предназначенных ограниченной группе узлов (адресатов);
- **broadcast** (широковещательный) – поток данных, предназначенных всем узлам (адресатам) в пределах одной локальной сети (ограниченный широковещательный поток); при необходимости возможен вариант передачи широковещательного трафика в соседнюю локальную сеть (направленный широковещательный поток) за счет специального конфигурирования маршрутизатора, объединяющего эти сети, позволяющего пропускать широковещательные пакеты;
- **anycast** (любой) – поток данных, предназначенных нескольким идентичным узлам, например DNS-серверам, при этом пакет будет доставлен ближайшему узлу (DNS-серверу).

Слайд 10

3.2. Адресация в IP-сетях

Типы адресов

В стеке протоколов TCP/IP используются три типа адресов:

- **физические (локальные) адреса**, используемые для адресации узлов (компьютеров, маршрутизаторов, коммутаторов, ...), примерами которых могут служить: MAC-адреса, если подсеть использует технологии Ethernet, Token Ring, FDDI, или IPX-адреса, если подсеть на основе технологии IPX/SPX;
- **сетевые (IP-адреса)**, используемые для идентификации узлов в пределах всей составной сети (подсети);
- **доменные имена** – символьные идентификаторы узлов, которыми оперируют пользователи.

Длина локального MAC-адреса, используемого наиболее широко из-за повсеместного распространения локальных сетей Ethernet, равна 6-и байтам.

При разработке стандарта локальной сети Ethernet было принято, что каждая сетевая карта и каждый встроенный сетевой интерфейс должны иметь уникальный номер – MAC-адрес, присвоенный при изготовлении.

Уникальность MAC-адреса достигается тем, что адреса назначаются *централизованно* комитетом IEEE, который распределяет между производителями интерфейсных карт (сетевых адаптеров) так называемые организационно уникальные идентификаторы (Organizationally Unique Identifier, OUI), отображаемые в первых трех байтах адреса, а в следующих трех байтах помещается порядковый номер сетевого адаптера, присваиваемый производителем.

При этом в первом байте адреса имеются два бита, определяющие тип и способ назначения адреса:

- **нулевой бит** (бит 0) указывает *тип адреса* (U/M – Unicast/Multicast): **0** – адрес назначения является **уникальным**, **1** – адрес назначения является **групповым** (тогда в последующих разрядах указывается адрес конкретной группы), или **широковещательным**, если все остальные разряды равны 1, то есть кадр адресован всем хостам в локальной сети;

- **первый бит** (бит 1) указывает *способ назначения адреса* (G/L – Global/Local): **0** – адрес является **глобальным** физическим адресом; **1** – адрес **локальный**, т.е. назначен администратором ЛВС и используется только в пределах этой сети.

Существует две формы записи MAC-адреса (форматы): **каноническая** и **бит-реверсная**. Последняя отражает последовательность передачи *битов* каждого байта по каналу связи.

Слайд 11

Классовая адресация

IP-адрес – идентификатор *сетевого соединения (сетевого интерфейса)*. Это означает, что один и тот же компьютер, соединенный с двумя сетями, имеет два IP-адреса: сеть 1 идентифицирует его по адресу IP1, а сеть 2 – по адресу IP2.

IP-адреса представляют собой 32-битовые идентификаторы, ориентированные на решение основной задачи протокола IP – маршрутизации. Для удобства представления IP-адресов используется цифровое их написание в виде 4-х десятичных чисел, разделенных точками, например: **192.171.153.60** .

Первоначально в Интернете была принята так называемая классовая адресация. Все IP-адреса разделены на 5 классов (от А до Е), но практическое применение находят в основном три первых класса: А, В и С. Класс D предназначен для задания группового адреса, а класс Е – не используется (был зарезервирован).

IP-адрес состоит из двух полей: поле «Номер сети», представляющий собой адрес физической сети (подсети), и поле «Номер узла», выделяющий в этой подсети конкретное устройство (хост).

Признаком принадлежности адреса к определённому классу служат первые биты адреса: если первый бит равен 0, то адрес принадлежит классу А, если первый бит равен 1, а второй – 0, то адрес принадлежит классу В и т.д.

Принадлежность адреса к тому или иному классу определяет размер сети:

- класс А соответствует большой сети с максимальным числом узлов ($2^{24} - 2$) = 16 777 214;
- класс В соответствует средней сети с числом узлов до 65534;
- класс С соответствует малой сети с числом узлов до $(2^8 - 2) = 254$.

Отметим, что **максимальное количество узлов** в сети определяется количеством двоичных разрядов n , отводимых под номер узла: $N_{\max} = 2^n - 2$, то есть исключаются два номера:

- **нулевой** (все разряды равны 0); адрес с нулевым значением номера узла означает *адрес сети*;
- **единичный** (все разряды равны 1); адрес с единичными значениями номера узла является *широковещательным* и означает передачу пакета всем узлам сети.

IP-адрес построен таким образом, чтобы поля «Номер сети» и «Номер узла» можно было бы выделить быстро, что особенно сказывается на эффективности маршрутизации (малые временные затраты на выделение адреса «Номер сети»).

Поскольку IP-адрес идентифицирует сетевое соединение, а не узел, то отсюда вытекает принципиальное ограничение: *если компьютер переносится из одной подсети в другую, он должен обязательно изменить IP-адрес.*

Слайд 12

Специальные адреса и маскирование

IP-адресация поддерживает **специальные адреса**, обращенные к множеству узлов и/или сетей и делящиеся на два класса:

- широковещательные (broadcast), обращенные ко всем;
- групповые (multicast), обращенные к заданному множеству объектов.

Адресная нотация при этом основывается на заполнении адресных полей нулями (обращение к данному объекту, this) или единицами (обращение ко всем объектам, all).

К специальным адресам относятся следующие.

Адрес 1 – "пустышка" или **неопределённый адрес**, используемый в инициализационной процедуре, когда рабочая станция не знает своего IP-адреса или хочет его согласовать; используется только как *адрес отправителя*, но никогда как адрес получателя.

Адрес 2 – адрес конкретного узла (XXX...X) в той же сети, что и узел-отправитель; применяется в случае, когда узел-отправитель не знает идентификатора сети, в которой работает, например при инициализации бездисковой рабочей станции, которая при включении вообще ничего не знает

ни о сети, ни о себе; используется только как *адрес получателя* и никогда как адрес отправителя.

Адрес 3 – адрес сети (но не узла).

Адрес 4 – **локальный** или **ограниченный широковещательный** адрес (limited или local broadcast address); используется, когда идентификатор сети по каким-либо причинам неизвестен; для использования не рекомендуется.

Адрес 5 – **прямой широковещательный** адрес (direct broadcast address), обращенный ко всем узлам данной сети.

Адрес 6 – **тестовый адрес**, в котором первый байт имеет значение 127, а оставшееся поле не специфицировано; используется для задач отладки и тестирования, не является адресом никакой сети, и маршрутизаторы никогда не обрабатывают его; также называется **адресом обратной петли** (*loopback address*), поскольку пакет с таким адресом, посланный на интерфейс loopback возвращается на тот же интерфейс, не выходя за пределы подсети.

Интерфейс loopback имеет несколько применений. Он может быть использован сетевым клиентским программным обеспечением, чтобы общаться с серверным приложением, расположенном на том же компьютере. Этот механизм полезен для тестирования служб, не подвергая их безопасности риску, как при удаленном сетевом доступе.

В стандартах Интернета определено несколько так называемых **автономных адресов**, рекомендуемых для автономного использования в пределах одной подсети и необрабатываемых маршрутизаторами:

- в классе А: 10.0.0.0 (1 сеть);
- в классе В: 172.16.0.0 – 172.31.0.0 (16 сетей);
- в классе С: 192.168.0.0 – 192.168.255.0 (256 сетей).

В качестве **группового адреса** используются адреса класса D. Групповая адресация в TCP/IP регламентируется входящим составной частью в IP протоколом *IGMP (Internet Group Management Protocol)*. Групповой адрес может объединять узлы из разных физических сетей путем использования в маршрутизаторах специальных протоколов групповой маршрутизации. Каждый узел может в любой момент подключиться к определенной адресной группе или выйти из нее.

Групповые адреса назначаются NIS и разделяются на два класса:

- **постоянные** – для непрерывно существующих групп (так называемые «всем известные адреса» – well-known addresses);
- **временные** – для организуемых на некоторый срок групп, которые существуют до тех пор, пока в группе сохраняется хотя бы один член (так называемые «временные адресные группы» – transient multicast groups).

Распространение групповых сообщений по сети ограничивается временем жизни TTL (time-to-live) IP-пакета.

Использование масок для IP-адресов

Маска представляет собой 32-разрядный двоичный код, содержащий в нескольких первых (старших) разрядах «единицы», а в остальных – «нули».

Количество единиц в маске определяет границу номера (идентификатора) сети. Другими словами, **единичные значения маски позволяют выделить из IP-адреса номер сети**, а оставшиеся младшие разряды **IP-адреса определяют номер узла в этой сети**.

Использование масок для IP-адресов позволяет *расширить адресное пространство* и сделать систему адресации более гибкой, не привязанной к классам **IP-адресов** (А, В или С).

Пример использования маски показан на слайде.

Часто использование маски указывается в виде: 116.165.42.35/**12**, где число **12** определяет количество единичных разрядов в маске для IP-адреса 116.165.42.35.

Слайд 13

Технология бесклассовой междоменной маршрутизации CIDR (Classless Inter-Domain Routing)

Использование масок переменной длины для IP-адресов позволяет не только расширить адресное пространство за счет увеличения количества номеров сетей, но и экономно выделять IP-адреса.

Например, если в какой-то небольшой сети находится десяток узлов, то очевидно, что неразумно выделять ей номер сети даже класса С, обеспечивающей нумерацию 254-х узлов. Гораздо более эффективным будет выделение для этой сети небольшого количества IP-адресов.

Для выделения ограниченного количества IP-адресов разработана технология *бесклассовой междоменной маршрутизации (CIDR – Classless Inter-Domain Routing)*, использующая *бесклассовую адресацию* и позволяющая гибко распределять IP-адреса.

Для реализации технологии CIDR необходимо, чтобы организация, распределяющая IP-адреса, имела в наличии непрерывный диапазон адресов. Это предоставляет возможность выделять сетям некоторое количество IP-адресов, имеющих *одинаковый префикс*, то есть одинаковые значения в нескольких старших разрядах.

На слайде показан пример, иллюстрирующий принцип выделения адресов из общего пула адресов для сетей разных размеров. Так, например:

- **пул 1** имеет префикс длиной 28 двоичных разрядов и 4 разряда под нумерацию узлов, что позволяет пронумеровать 16 узлов небольшой сети;
- **пул 2** с префиксом длиной 26 разрядов и с 6-ю разрядами под нумерацию узлов позволяет пронумеровать 64 узла;
- **пул 3** с префиксом длиной 22 разряда позволяет пронумеровать 1024 узла.

При таком выделении адресов необходимо, чтобы:

- количество выделяемых адресов было кратно степени двойки;
- начальная граница выделяемого пула адресов была кратна требуемому количеству узлов.

Благодаря технологии CIDR имеется возможность нарезать блоки адресов в соответствии с действительными потребностями каждой сети.

Слайд 14

Формирование подсетей в большой сети

Технология CIDR, использующая маскирование сетевых адресов, позволяет большую сеть некоторой организации разбить на подсети, каждая из которых объединяет, например, пользователей одного подразделения (отдела, лаборатории).

Очевидно, что количество подсетей и хостов в каждой подсети с заданной длиной маски зависит от размера исходной большой сети.

В таблице представлены варианты формирования подсетей одинакового размера, число которых зависит от количества хостов в подсети.

Следует отметить, что общее количество хостов в сети уменьшается с увеличением количества подсетей.

Слайд 15

Адресация подсетей разных размеров в рамках сети 185.68.1.0/24

Большая сеть может быть разбита на подсети разного размера.

На слайде иллюстрируется один из возможных вариантов разбиения большой сети 185.68.1.0/24 на 6 подсетей, причем все подсети разного размера за счет присвоения им разных масок: от сети со 126 узлами (хостами) с маской /25 до подсети с двумя узлами с маской /30.

Таким образом, в рамках сети 185.68.1.0/24 сформировано 6 подсетей разного размера с разными масками с максимальным суммарным числом узлов, равным 240.

Слайд 16

Технология NAT

NAT (Network Address Translation) – сетевая технология, направленная на решение проблемы ограниченного диапазона IP-адресов в IPv4 и повышение безопасности сети за счет:

- ограничения доступа извне к ресурсам внутренней сети при сохранении возможности выхода в публичную (внешнюю) сеть;
- сокрытия внутренних сервисов, хостов, серверов.

Изменение адреса при передаче пакета из внутренней сети во внешнюю и обратно осуществляется пограничным маршрутизатором, в котором реализована технология NAT. Это изменение выполняется с использованием NAT-таблицы, устанавливающей соответствие между глобальными IP-адресами внешней сети и локальными IP-адресами внутренней сети. В некоторых реализациях NAT-

таблица дополнительно может содержать номера портов, привязанных к IP-адресам.

Существует несколько типов технологии NAT, основными из которых являются:

- статический (Static) NAT, когда один локальный адрес соответствует одному глобальному адресу;
- динамический (Dynamic) NAT, когда локальному адресу соответствуют разные глобальные адреса;
- Port Address Translation (PAT) или NAT Overload (перегруженный NAT), когда сопоставление локальных и глобальных адресов осуществляется с использованием портов.

Статический NAT обычно применяется для отображения незарегистрированного IP-адреса на зарегистрированный IP-адрес и используется в сети, когда необходимо, чтобы IP-адрес всегда был доступен из глобальной сети.

Динамический NAT отображает незарегистрированный IP-адрес внутренней сети на зарегистрированный адрес из пула зарегистрированных IP-адресов внешней сети, причем зарегистрированный адрес может меняться в зависимости от доступного в данный момент IP-адреса в пуле адресов.

Перегруженный NAT (NAT Overload) или PAT динамически отображает несколько незарегистрированных IP-адресов в один зарегистрированный IP-адрес, используя при этом различные порты.

Слайд 17

Распределение IP-адресов

Способы распределения IP-адресов:

- **централизованное распределение**, реализуемое в соответствии с иерархией, показанной на слайде;
- **автоматизированное распределение** (назначение компьютерам) IP-адресов внутри подсети на основе протокола DHCP.

DHCP (Dynamic Host Configuration Protocol) – протокол динамической конфигурации хостов, предназначенный для автоматического получения сетевыми устройствами (компьютерами) IP-адреса и некоторых дополнительных параметров, необходимых для работы в сети TCP/IP.

Протокол DHCP предоставляет три способа распределения IP-адресов.

1. **Ручное статическое распределение**, при котором сетевой администратор назначает физическому адресу (MAC-адресу в сетях Ethernet) клиентского компьютера определённый IP-адрес. При этом роль DHCP-сервера сводится к централизованному хранению сведений об адресах.

2. **Автоматическое статическое распределение**, при котором компьютеру предоставляется на постоянное использование свободный IP-адрес из имеющегося диапазона адресов.

3. **Автоматическое динамическое распределение**, которое отличается от предыдущего способа лишь тем, что IP-адрес выдаётся компьютеру на определённый срок – *срок аренды адреса*, по истечении которого адрес считается свободным, а клиент при необходимости должен запросить новый адрес. Таким образом, один и тот же IP-адрес в разные интервалы времени может быть предоставлен разным компьютерам.

Для корректной работы в сети компьютеру, кроме IP-адреса, требуются другие конфигурационные параметры, такие как:

- маска подсети;
- IP-адрес маршрутизатора по умолчанию;
- IP-адреса DNS-серверов;
- доменное имя компьютера и т.д.

Эти параметры включаются в DHCP-сообщение и передаются вместе с IP-адресом.

Слайд 18

Протокол DHCP

Протокол DHCP был принят в 1993 году и в настоящее время используется в большинстве сетей TCP/IP.

Протокол DHCP работает на *прикладном уровне* OSI-модели по схеме «клиент-сервер», используя порт 67/UDP или 68/UDP. Для автоматической конфигурации компьютер-клиент при включении обращается к серверу DHCP и получает от него **IP-адрес** и **ряд конфигурационных параметров**.

DHCP-сервер – это компьютер, который распределяет IP-адреса и следит за тем, чтобы один и тот же IP-адрес не был выдан двум клиентам. Для получения IP-адреса клиент и сервер обмениваются между собой DHCP-сообщениями в режиме запрос-ответ. Назначение DHCP-сообщений представлено в таблице на слайде.

Получение IP-адреса выполняется за 4 шага.

1. Компьютер клиента для нахождения DHCP-сервера формирует сообщение DISCOVER с широковещательным MAC-адресом FF:FF:FF:FF:FF:FF, которое принимают все компьютеры в сети. Использование широковещательного адреса обусловлено тем, что клиент при включении не знает, где находится DHCP-сервер.

2. В ответ DHCP-сервер посылает клиенту сообщение OFFER с IP-адресом.

3. Клиент подтверждает получение IP-адреса, посылая сообщение (запрос) REQUEST с тем же IP-адресом.

4. DHCP-сервер в ответ посылает подтверждение ACK о возможности использования клиентом данного IP-адреса.

Такую процедуру получения IP-адреса по протоколу DHCP обозначают аббревиатурой **DORA** – по первым буквам передаваемых DHCP сообщений: **DISCOVER, OFFER, REQUEST, ACK**.

Кроме перечисленных сообщений могут использоваться и другие сообщения:

- **NACK** – на запрос клиентом IP-адреса в сообщении REQUEST запрещает его использование;
- **RELEASE** – клиент освобождает IP-адрес;
- **DECLINE** – клиент отказывается от предлагаемого IP-адреса;
- **INFORM** - используется клиентом, имеющим IP-адрес, для получения дополнительных конфигурационных параметров;
- **FORCERENEW** – требование освободить IP-адрес (практически не используется).

Может возникнуть вопрос: а зачем нужен 4-й шаг? Разве трёх шагов недостаточно?

Такая необходимость в 4-м шаге может возникнуть в следующих ситуациях:

- 1) возникла **ошибка** при передаче IP-адреса от DHCP-сервера клиенту, которая в случае 3-х шагов не будет обнаружена и клиент будет использовать этот ошибочный IP-адрес; наличие 4-го шага позволяет DHCP-серверу выслать запрет на использование IP-адреса в виде сообщения NACK
- 2) в сети имеется **два и более DHCP-сервера**.

Компьютеру IP-адрес выделяется DHCP-сервером на ограниченное время – **время аренды (lease time)**, которое может составлять от нескольких минут до нескольких дней. По истечении времени аренды IP-адрес освобождается и может передан другому компьютеру.

При необходимости аренда IP-адреса может быть продлена клиентом. Для этого клиент по истечении половины времени аренды пересылает серверу сообщение REQUEST, в котором указывает этот IP-адрес. В ответ сервер высылает сообщение ACK, подтверждающее возможность дальнейшего использования этого IP-адреса.

При выключении компьютера операционная система автоматически отправляет DHCP-серверу сообщение RELEASE, извещающее об освобождении IP-адреса, который может быть передан другому компьютеру. При некорректном выключении компьютера и при сбоях в операционной системе IP-адрес сохраняется за компьютером до тех пор, пока не закончится время аренды.

DHCP-сервер и клиенты, запрашивающие IP-адрес, обычно располагаются в одной и той же подсети. Если же DHCP-сервер находится в другой подсети,

отделенной от клиента маршрутизатором, то DHCP-сообщение DISCOVER может не дойти до сервера, поскольку *маршрутизатор не пропускает широковещательный трафик*. Для решения этой проблемы может использоваться специальная конфигурация маршрутизатора – **DHCP-Relay**, которая позволяет пропускать (транслировать) широковещательный трафик, но не весь, а только DHCP-сообщений. В этом случае широковещательное DHCP-сообщение DISCOVER будет доставлено от клиента к DHCP-серверу, который находится в другой подсети, а DHCP-сервер вышлет IP-адрес клиенту.

Слайд 19

Протоколы ARP и RARP (InARP)

Определение физического адреса устройства (MAC-адреса) по его IP-адресу и наоборот, IP-адреса по MAC-адресу, решают три протокола:

IP-адресу и наоборот, IP-адреса по MAC-адресу, решают три протокола:

- **ARP** (*Address Resolution Protocol*) – протокол разрешения адресов, предназначенный для определения MAC-адреса по IP-адресу;
- **RARP** (*Reverse Address Resolution Protocol*) – протокол обратного определения адреса: IP-адреса по MAC-адресу;
- **InARP** (*Inverse Address Resolution Protocol*) – инверсный протокол для получения сетевых адресов других рабочих станций по адресам канального уровня.

Протокол ARP в режиме запрос-ответ позволяет автоматически определить MAC-адрес компьютера по его IP-адресу. Для этого протокол ARP поддерживает в каждом узле (сетевом адаптере или порту маршрутизатора) **ARP-таблицу**, содержащую:

- IP-адрес;
- MAC-адрес;
- тип записи (динамический, статический).

Статические записи создаются вручную и существуют, пока соответствующий узел (компьютер или маршрутизатор) не будет выключен.

Динамические записи создаются протоколом ARP как *по собственным ARP-запросам*, так и путем *извлечения из широковещательных запросов IP- и MAC-адресов отправителя*. Динамическая запись имеет «срок жизни», после которого она удаляется. Срок жизни записи устанавливается администратором сети или принимается по умолчанию и может составлять от десятков секунд до нескольких часов.

По ARP-таблице узел может определить физический адрес (MAC-адрес) узла назначения, находящегося *в этой же сети*, по известному IP-адресу и указать его в заголовке кадра канального уровня.

Если в ARP-таблице отсутствует запись для некоторого IP-адреса, то узел формирует *широковещательное* сообщение – **ARP-запрос** (ARP-request), в котором запрашивает физический адрес узла назначения с заданным IP-адресом.

Все узлы сети принимают этот запрос, однако лишь один узел, IP-адрес которого совпадает с указанным в ARP-запросе, отвечает на него, высылая **ARP-ответ** (ARP-reply), формат которого такой же, как у ARP-запроса, со своим MAC-адресом *непосредственно* узлу, приславшему ARP-запрос. *Отправитель извлекает MAC-адрес и вставляет его в поле «адрес назначения» формируемого для передачи по локальной сети кадра.* Полученный отправителем ARP-запроса MAC-адрес кэшируется в ARP-таблице.

Формат ARP-запроса (ответа), представленный на слайде, содержит 9 полей.

Протокол ARP в модели OSI располагается *между канальным и сетевым уровнем* (уровень 2,5). Пакеты ARP вкладываются непосредственно в **кадры локальной сети** и не передаются через маршрутизаторы, поскольку содержат широковещательный адрес. Следовательно, с помощью протокола ARP можно узнать только MAC-адреса компьютеров, которые находятся в одной подсети (в локальной сети).

Отправитель также включает в ARP запрос свой IP-адрес и свой MAC-адрес, которые могут быть занесены всеми компьютерами сети в ARP-таблицы.

Каждый компьютер локальной сети может формировать специальные ARP-запросы. Примером такого запроса может служить так называемый «**добровольный**» ARP-запрос (*Gratuitous ARP*), с помощью которого можно:

- *оповестить* все компьютеры сети об изменении или появлении у данного компьютера нового IP-адреса;
- *проверить* возможность использования данного IP-адреса другим компьютером: если на такой ARP-запрос пришел ответ, то это значит, что в сети есть компьютер с таким IP-адресом.

Таким образом, ARP-протокол разрешения адресов:

- позволяет *по IP-адресу определить MAC-адрес* компьютера в сети;
- работает в режиме *запрос-ответ*;
- запрос отправляется на *широковещательный адрес* и его получают все компьютеры в сети, а отвечает только тот компьютер, который узнал свой IP-адрес и в ответ он вкладывает искомый MAC-адрес.

Наиболее типичная ситуация, при которой возникает необходимость использования протокола ARP, следующая. В сети, объединяющей несколько локальных сетей (подсетей) с помощью маршрутизаторов, продвижение пакетов от узла, находящегося в одной подсети, к узлу, находящемуся в другой подсети, осуществляется на основе старшей части IP-адреса, то есть на основе номера сети. После того, как пакет поступит в *конечный маршрутизатор*, к которому подсоединена локальная сеть, в которой находится получатель кадра, необходимо этот пакет упаковать в кадр, а в качестве канального адреса узла назначения указать его MAC-адрес. Маршрутизатор просматривает свою ARP-таблицу и, если не находит соответствующего IP-адреса, формирует широковещательный ARP-запрос, посылает его в локальную сеть и ожидает

ARP-ответа. Если в сети нет компьютера с указанным в ARP-запросе IP-адресом, то ARP-ответа не будет, и протокол IP уничтожит все пакеты, направляемые по этому адресу.

Посмотреть свою ARP таблицу на компьютере можно с помощью команды **arp -a**.

RARP (*Reverse Address Resolution Protocol*) — *обратный протокол преобразования адресов, предназначенный для определения IP-адреса по физическому адресу.*

Протокол используется в случае, если узел – бездисковая рабочая станция, у которой только что включили питание и она не только ничего не знает о себе и окружающих, но и не может произвести дистанционную загрузку операционной системы, которая хранится на сетевом диске. В этом случае формируется групповой RARP-запрос со своим физическим адресом (при этом узел может даже не знать адреса сервера). Сервер принимает запрос и просматривает свои таблицы, либо перенаправляет его. В сети находится по меньшей мере один обслуживающий такие запросы сервер (RARP-сервер), который распознает запрос от рабочей станции, выбирает из некоторого списка свободный IP-адрес и шлет этому узлу сообщение с необходимой информацией:

- динамически выделенный узлу IP-адрес;
- свой физический адрес;
- IP-адрес и т.д.

После обнаружения найденный адрес отсылается на запросивший его узел. Другие станции также могут «слышать» этот диалог и локально сохранить эту информацию в своих ARP-таблицах.

Поскольку при таком механизме отказ RARP-сервера очень критичен в том смысле, что без его услуг не заработает целый ряд рабочих станций, то обычно сеть конфигурируется так, чтобы протокол RARP поддерживали несколько серверов в сети.

RARP позволяет разделять IP-адреса между нечасто используемыми хост-узлами. После использования каким-либо узлом IP-адреса он может быть освобождён и выдан другому узлу.

RARP является дополнением к ARP.

InARP (*Inverse Address Resolution Protocol, Inverse ARP*) — протокол, предназначенный для получения сетевых адресов (и *не только IP-адресов*) других рабочих станций по их адресам канального уровня. В основном используется в сетях Frame Relay и ATM.

InARP можно рассматривать как расширение ARP.

Таким образом, RARP, как и InARP, предназначен для определения адреса сетевого уровня по адресу канального уровня. Отличие состоит только в том, что RARP-протокол используется для получения сетевого адреса *самой станции*

отправителя, в то время как InARP-протокол запрашивает сетевой адрес по физическому адресу другой станции.

Отметим, что ARP запросы и ответы размещаются в поле данных кадра, что позволяет рассматривать протокол ARP как протокол OSI-модели промежуточного уровня 2,5, расположенного между сетевым и канальным уровнями.

Слайд 20

Схема реализации протокола ARP

На слайде показана схема реализации протокола ARP в случае отсутствия в **ARP-таблице** пограничного маршрутизатора **Мш** записи о соответствии MAC-адреса конкретного компьютера, которому должен быть направлен пакет, IP-адресу назначения, указанному в заголовке пакета.

Положим, что после прохождения по сети Интернет в пограничный маршрутизатор **Мш** локальной сети LAN Ethernet поступает пакет с IP-адресом назначения **195.67.8.14**. Для передачи этого пакета в локальную сеть его необходимо упаковать в соответствующий кадр канального уровня, в заголовке которого должен быть указан MAC-адрес конкретного компьютера. Поскольку в **ARP-таблице Мш** отсутствует запись, устанавливающая соответствие MAC-адреса IP-адресу назначения $I_{pn}=195.67.8.14$, формируется *широковещательный ARP-запрос*, который направляется в локальную сеть. Компьютер с номером 5 с IP-адресом **195.67.8.14** откликается на запрос и посылает маршрутизатору ARP-ответ со своим MAC-адресом **a1-24-d2-1c-05-44**. Маршрутизатор формирует в своей **ARP-таблице** новую запись, устанавливающую соответствие MAC-адреса **a1-24-d2-1c-05-44** IP-адресу **195.67.8.14**.

Теперь маршрутизатор **Мш** может вставить полученный MAC-адрес в заголовок формируемого кадра и послать его в локальную сеть.

Слайд 21

Протокол DNS (Domain Name System)

Доменное имя – символьное имя компьютера, состоящее из нескольких частей.

Совокупность имен, у которых несколько старших составных частей совпадают, образуют **домен (domain) имен**.

В стеке TCP/IP применяется система доменных имен с *иерархической древовидной структурой*, допускающей использование в имени произвольного количества составных частей.

Примерами доменных имён организаций являются:

- com – коммерческие организации;
- edu – образовательные организации;

- gov – правительственные организации;
- org – некоммерческие организации;
- net – организации поддержки сетей.

Соответствие между доменными именами и IP-адресами может устанавливаться как средствами локального узла, так и средствами централизованной службы, реализуемой системой доменных имён.

Система доменных имен DNS – централизованная служба, основанная на распределенной базе отображений «доменное имя – IP-адрес». DNS предназначена для замены цифровых адресов на символные, более понятные пользователям.

Служба DNS использует в своей работе протокол типа «клиент-сервер», в котором определены такие понятия как **DNS-сервер**, поддерживающий распределенную базу отображений, и **DNS-клиент**, обращающийся к DNS-серверу с запросом. DNS-сервер использует текстовые файлы формата «IP-адрес – доменное имя».

Служба DNS является распределенной. Каждый DNS-сервер хранит имена следующего уровня иерархии и, кроме таблицы отображений имен, содержит ссылки на DNS-серверы своих поддоменов, что упрощает процедуру поиска.

Для ускорения поиска IP-адресов в DNS-серверах применяется кэширование проходящих через них ответов на определенное время – от нескольких часов до нескольких дней.

Иерархическая организация DNS предполагает использование четырех типов DNS-серверов.

1. **Рекурсивный DNS-сервер (Resolver)** – сервер, который формирует запросы к другим DNS-серверам, пока не найдёт IP-адрес по запросу DNS-клиента. В качестве ответа возвращает клиенту найденный IP-адрес, либо сообщение об ошибке, если IP-адрес не найден. Обычно для надежности создаются 2 рекурсивных DNS-сервера: *основной* и *запасной*. Например, основной публичный рекурсивный DNS-сервер от Google имеет IP-адрес 8.8.8.8, а запасной – 8.8.4.4.
2. **Корневой (Root) DNS-сервер** обслуживается *Интернет-корпорацией по управлению доменными именами и IP-адресами* (ICANN - Internet Corporation for Assigned Names and Numbers). В настоящее время в Интернете 13 корневых DNS-серверов (8 в США, 3 в Европе и 2 в Азии), копии которых распределены по всему миру. В России копии есть в Москве, Екатеринбурге и Новосибирске. Основная задача корневых DNS-серверов – поддерживать каталоги для существующих доменных зон. Другими словами, они знают адреса TLD-серверов, отвечающих за конкретные доменные зоны: «.ru», «.com», «.net» и т.д. То есть, если нужно найти IP-адрес домена «itmo.ru», он вернёт IP адрес TLD-сервера, отвечающего за зону «.ru».
3. **TLD (Top Level Domain)** – сервер домена верхнего уровня – хранит каталог с адресами *авторитативных* серверов своей зоны и поддерживается *Административным органом по присвоению адресов* –

IANA (Internet Assigned Numbers Authority), которая входит в ICANN. TLD-сервер знает на каком авторитативном сервере хранится информация о любом домене из его зоны.

4. **Авторитативный (Authoritative) DNS-сервер** хранит всю информацию о конкретном домене. Это не только IP адрес, но и другие записи разных типов, обозначаемых заглавными буквами:

- А — адрес хоста (IPv4 address);
- AAAA — IPv6 адрес (IPv6 address);
- MX — имена почтовых серверов (Mail eXchange);
- NS — сервер домена (Name Server);
- TXT — текстовые записи (Text), SPF.

Там же хранится информация об организации, которая осуществила регистрацию домена. Эти данные в свободном доступе и их можно получить через онлайн-сервисы.

Авторитативный DNS-сервер:

- быстро реагирует на запросы;
- не отвечает на рекурсивные запросы;
- не кэширует результаты запросов.

Поиске IP-адреса сайта по его доменному имени выполняется в несколько этапов.

1. DNS-клиент, желающий послать сообщение по *известному доменному имени*, проверяет наличие соответствующего IP-адреса в локальном кеше. Если IP-адрес найден, то он используется в качестве адреса назначения в заголовке формируемого сообщения (пакета). На этом работа DNS заканчивается.
2. Если в кеше не найден соответствующий IP-адрес, запрос отправляется на внешний **рекурсивный DNS-сервер**, IP-адрес которого указан в настройках сетевого адаптера компьютера, либо может быть получен автоматически от Интернет-провайдера по протоколу DHCP. (*IP-адрес рекурсивного DNS-сервера можно узнать с помощью команды «nslookup» в командной строке Windows*).
3. **Рекурсивный DNS-сервер** также ищет в своей базе IP-адрес, соответствующий *доменному имени*, и если не находит, то последовательно:
 - формирует запрос к одному из **корневых DNS-серверов** для определения *IP-адреса TLD-сервера*, который отвечает за доменную зону верхнего (первого) уровня;
 - посылает запрос к **TLD-серверу доменной зоны**, чтобы получить *IP-адрес авторитативного DNS-сервера*, содержащего полную информацию о запрашиваемой доменной зоне;
 - формирует запрос к **авторитативному DNS-серверу** о запрашиваемой доменной зоне для получения искомого IP-адреса, соответствующего *доменному имени*;

- если искомый IP-адрес найден, то он отсылается DNS-клиенту; в противном случае клиенту отправляется сообщение DNS_PROBE_FINISHED_NXDOMAIN (адрес не найден).
4. DNS-клиент, получив IP-адрес кеширует его для дальнейшего использования.
 5. Используя этот IP-адрес DNS-клиент посылает сообщение.

Эффективность и результативность протокола DNS обеспечивается за счет:

- распределённого администрирования;
- распределённого хранения данных;
- кэширования информации;
- иерархической структуры;
- резервирования.

Протокол DNS в качестве транспорта может использовать TCP или UDP и порт 53. Обычно небольшие запросы и ответы формируются в виде одной UDP-дейтаграммы. TCP используется в том случае, если размер данных превышает 512 байт.

Слайд 22

3.3. Фрагментация IP-пакетов

Прозрачная и сквозная фрагментация

В объединяемых сетях разных технологий допустимая максимальная длина пакетов (**Maximum Transfer Unit, MTU**) различна и варьируется от 53 байт в ATM-сетях до 65 535 байт в IP-сетях. При объединении таких сетей возникает проблема, связанная необходимостью разбиения большого пакета при его передаче через сеть с меньшей допустимой длиной пакета. Если пакет проходит через последовательность сетей и попадает в сеть, у которой значение MTU оказывается меньше размера пакета, пограничный маршрутизатор разбивает пакет на две или более части.

Процесс разбиения длинного пакета на более короткие называется **фрагментацией**, а соответствующие короткие пакеты – **фрагментами**. При фрагментации каждый новый пакет получает свой IP-заголовок (20 байт), что увеличивает накладные расходы. После прохождения фрагментов через соответствующую сеть необходимо восстановить исходный пакет из фрагментов.

Фрагментация в сетях может быть реализована двумя способами:

- прозрачная фрагментация;
- сквозная фрагментация.

Принцип реализации **прозрачной фрагментации** на примере передачи длинного пакета от узла А к узлу В через две подсети (Ethernet и ATM) с меньшим значением MTU показан на слайде.

Подсети с разными MTU имеют шлюзы – специализированные маршрутизаторы, предоставляющие интерфейсы для связи с другими сетями. Если на такой шлюз приходит пакет слишком большого размера, он разбивается на фрагменты в соответствии с принятым в данной сети значением MTU. Каждый фрагмент адресуется одному и тому же выходному шлюзу, который восстанавливает из этих фрагментов исходный пакет. Таким образом, прохождение данных через сети (подсети) с маленькими значениями MTU оказывается прозрачным для пользователей.

Прозрачная фрагментация обладает простотой, но при этом имеет ряд существенных недостатков:

- выходной шлюз должен собрать все фрагменты для восстановления исходного пакета, для чего в заголовках фрагментов необходимо иметь дополнительную информацию, например, номер фрагмента и признак последнего фрагмента;
- все фрагменты одного пакета должны покидать подсеть через один и тот же шлюз, что снижает эффективность маршрутизации;
- появляются дополнительные накладные расходы на фрагментацию и дефрагментацию, что снижает производительность сети и увеличивает время доставки пакетов.

Сквозная фрагментация является альтернативной по отношению к прозрачной фрагментации и состоит в отказе от восстановления пакета из фрагментов в каждой подсети. Пакет разбивается на фрагменты сразу же в узле-отправителе А или в шлюзе Ш₁ сети. Эти фрагменты передаются по сети как самостоятельные пакеты независимо друг от друга и собираются только в конечном шлюзе Ш_N или узле-получателе В.

Недостатками такого способа фрагментации являются следующие:

- необходимо, чтобы каждый узел (или шлюз) могли восстанавливать пакеты из фрагментов;
- возрастают накладные расходы на передачу данных, так как каждый фрагмент должен иметь заголовок, который сохраняется на протяжении всего пути, что снижает пропускную способность сети;
- необходимо иметь информацию о том, какие значения MTU имеют подсети, через которые проходит путь передачи данных, чтобы задать размер фрагментов.

Для того чтобы правильно восстановить исходный пакет из фрагментов необходимо иметь эффективную систему нумерации фрагментов. Одна из таких систем основана на понятии **элементарного фрагмента**, имеющего небольшой размер, достаточный для его передачи через любую подсеть. Например, длина элементарного фрагмента может быть равна 8 байтам. Исходный пакет разбивается на множество элементарных фрагментов одинаковой длины, кроме последнего, который может быть короче. Фрагменты, формируемые в некоторой подсети и называемые **межсетевыми пакетами**, могут состоять из нескольких элементарных фрагментов, число которых определяется значением MTU, принятым для данной подсети. Заголовок таких фрагментов должен содержать:

- номер исходного пакета (ИП);
- номер первого элементарного фрагмента (нумерация начинается с нуля), содержащегося в нём, который в заголовке IP-пакета называется *смещением фрагмента* (СФ);
- признак конца (ПК) пакета.

Поскольку размер элементарного фрагмента выбирается таким образом, чтобы он мог пройти через любую сеть, дальнейшая фрагментация межсетевого пакета не составляет проблемы.

Слайд 23

3.4 Транспортные протоколы TCP/IP

Общие принципы

Транспортные протоколы TCP и UDP стека протоколов TCP/IP обеспечивают передачу данных между любой парой *прикладных процессов*, выполняющихся в сети, и предоставляют интерфейс для протокола IP путем демультиплексирования нескольких процессов, использующих в качестве адресов транспортного уровня порты. Для каждого прикладного процесса (ПП) (приложения), выполняемого в компьютере, может быть сформировано *несколько точек входа*, выступающих в качестве *транспортных адресов*, называемых **портами**.

Существуют два способа присвоения порта приложению:

- **централизованный** (присвоенные или назначенные номера от 0 до 1023), использующий стандартные номера, присвоенные общедоступным службам (приложениям), например: FTP – 21, telnet – 23, SMTP – 25, DNS – 53, HTTP – 80.
- **локальный** (динамические номера от 1024 до 65535), предоставляющий произвольный номер из списка свободных номеров при поступлении запроса от приложения пользователя.

Динамические номера портов приложений являются уникальными в пределах каждого компьютера, но могут совпадать с номерами портов в других компьютерах. Различие между ними определяется только различием интерфейсов каждого из компьютеров, задаваемых IP-адресами. Таким образом, пара «**IP-адрес; номер порта**», называемая **сокетом** (socket), однозначно определяет прикладной процесс в сети.

Номера UDP- и TCP-портов в пределах одного и того же компьютера могут совпадать, хотя и идентифицируют разные приложения. Поэтому при записи номера порта обязательно указывается тип протокола транспортного уровня, например 2345/TCP и 2345/UDP. В некоторых случаях, когда приложение может обращаться по выбору к протоколу UDP или TCP, ему могут быть назначены одинаковые номера UDP- и TCP-портов, например DNS-приложению назначен номер 53 – 53/UDP и 53/TCP.

Слайд 24

Протокол UDP

UDP – транспортный протокол, обеспечивающий передачу данных в виде *дейтаграмм* между любой парой *прикладных процессов*, выполняющихся в сети, *без установления соединения*. Дейтаграммы состоят из 8-байтового заголовка, за которым следует поле данных. Заголовок UDP-дейтаграммы показан на слайде.

Наиболее широко UDP используется при выполнении клиент-серверных приложений (типа запрос-ответ).

При этом UDP:

- не контролирует поток,
- не контролирует ошибки в процессе передачи данных,
- не повторяет передачу после получения испорченной дейтаграммы.

Примерами приложений, использующих протокол UDP для передачи данных, являются DHCP, DNS, SNMP.

В некоторых случаях на одном конечном узле может выполняться несколько копий одного и того же приложения. Возникает вопрос: каким образом различаются эти приложения?

Для этого рассмотрим на простом примере процесс формирования запроса и процедуру обращения DNS-клиента к DNS-серверу, когда на одном компьютере запущены два DNS-сервера, причём оба используют для передачи своих данных транспортный протокол UDP. Для того чтобы различать DNS-серверы, им присваиваются разные IP-адреса – IP1 и IP2, которые вместе с номером порта образуют два разных сокета: «UDP-порт 53, IP1» и «UDP-порт 53, IP2».

Рисунок на слайде иллюстрирует процесс формирования DNS-клиентом запроса к DNS-серверу.

DNS-запрос транспортном уровне стека протоколов TCP/IP передаётся протоколу UDP, который вкладывает этот запрос в UDP-дейтаграмму и указывает в заголовке порт назначения 53/UDP. Затем UDP-дейтаграмма передаётся на межсетевой уровень, где она вкладывается в IP-пакет, заголовок которого содержит «IP-адрес: IP2». IP-пакет, в свою очередь, передаётся на уровень «межсетевой интерфейс», где он помещается в кадр канального уровня с соответствующим заголовком канального уровня (ЗКУ). Этот кадр передаётся по сети к компьютеру, содержащему два DNS-сервера.

В этом компьютере протокол канального уровня (ПКУ) снимает заголовок ЗКУ и передаёт содержимое кадра на межсетевой уровень протоколу IP, который, в свою очередь, извлекает содержимое (UDP-дейтаграмму) из IP-пакета. Дальнейшие манипуляции с передаваемыми данными отличаются от принципов, заложенных в многоуровневую модель иерархии протоколов. Вместо того чтобы просто передать UDP-дейтаграмму, находящуюся в поле данных IP-пакета, транспортному уровню, *IP-протокол присоединяет к UDP-дейтаграмме* так называемый **псевдозаголовок**, содержащий среди прочего IP-адреса отправителя и получателя. Таким образом, протокол UDP, имея IP-адрес

и порт назначения, однозначно определяет, что содержимое поля данных (то есть DNS-запрос), должно быть передано приложению «DNS-сервер 2».

Псевдозаголовок используется прежде всего для *расчета контрольной суммы* при отправлении и при получении UDP-дейтаграмм и TCP-сегментов.

Слайд 25

Псевдозаголовок протоколов UDP и TCP

Как сказано выше, при передаче данных от нижележащего межсетевого уровня на транспортный уровень в заголовки UDP-дейтаграмм и TCP-сегментов включается псевдозаголовок, который располагается перед основным заголовком транспортного уровня. Таким образом, блок данных транспортного уровня (UDP-дейтаграмма или TCP-сегмент) будет содержать:

- псевдозаголовок длиной 12 байт (три 32-разрядных слова);
- заголовок длиной 8 байт для UDP-дейтаграммы или 20 и более байт для TCP-сегмента;
- данные.

Псевдозаголовок, формат которого показан на слайде, содержит:

- IP-адрес отправителя;
- IP-адрес получателя;
- нулевое поле, не используемое и заполненное нулями;
- поле «Протокол», содержащее номер протокола транспортного уровня: 17 для протокола UDP и 6 для протокола TCP;
- длина UDP-дейтаграммы или TCP-сегмента.

Включение псевдозаголовка в контрольную сумму блока данных транспортного протокола помогает обнаружить неверно доставленные пакеты за счёт двойной проверки, выполняемой протоколом IP и протоколами транспортного уровня. Кроме того, передача IP-адресов транспортному уровню позволяет однозначно разрешить ситуацию, когда две копии одного и того же приложения используют одинаковый номер порта.

Узел-отправитель при формировании UDP-дейтаграммы или TCP-сегмента рассчитывает контрольную сумму сегмента с учётом псевдозаголовка. Если в процессе расчета контрольная сумма оказывается равной нулю, то поле «Контрольная сумма» заполняется единицами.

Если при передаче UDP-дейтаграммы это поле заполнено нулями, то это означает, что контрольная сумма для данной дейтаграммы не рассчитывается и не проверяется, что *позволяет уменьшить время ее доставки*. Для TCP-сегментов контрольная сумма всегда рассчитывается, что гарантирует надежность доставки.

При передаче по сети псевдозаголовок не включается в дейтаграмму (сегмент), что позволяет уменьшить накладные расходы и, соответственно, повысить эффективную скорость передачи пользовательских данных. В узле-получателе протокол IP формирует псевдозаголовок и вставляет его в поступивший сегмент и передаёт транспортному уровню.

Слайд 26Протокол ТСР: принципы реализации

1. Протокол ТСР обеспечивает **надежную передачу** данных между прикладными процессами за счет установления логических соединений между взаимодействующими процессами.

Логическое соединение между двумя прикладными процессами идентифицируется парой сокетов (IP-адрес, номер порта), каждый из которых описывает один из взаимодействующих процессов.

2. В процессе установления логического соединения отправитель и получатель **согласовывают параметры процедуры обмена** данными, основными среди которых являются:

- *начальный порядковый номер байта* ISN (Initial Sequence Number), представляющий собой случайное число в интервале $(0; 2^{32})$;
- *максимальный размер передаваемых сегментов*;
- *максимальный объем данных*, который получатель разрешает передать отправителю без подтверждения (максимальный размер окна).

3. Информация, поступающая к протоколу ТСР в рамках логического соединения от протоколов более высокого уровня, рассматривается протоколом ТСР как *неструктурированный поток байтов* и заносится в буфер. Для передачи на сетевой уровень из буфера вырезается **сегмент**, не превосходящий 64 Кбайт (максимального размера IP-пакета). На практике обычно длина сегмента ограничивается значением 1460 байтами, что позволяет поместить его в поле данных (1500 байт) кадра Ethernet с заголовками ТСР (20 байт) и IP (20 байт).

Соединение ТСР ориентировано на *дуплексную передачу*, причем если одна из сторон не имеет данных для передачи, она формирует и передает положительные квитанции в рамках механизма «скользящего окна», содержащие только заголовок сегмента (без поля данных).

4. **Управление потоком** данных в протоколе ТСР осуществляется с использованием механизма **скользящего окна переменного размера**.

5. При передаче сегмента узел-отправитель включает таймер и ожидает подтверждения в течение **тайм-аута**.

6. В протоколе ТСР используются *только положительные квитанции* – подтверждения АСК (от англ. Acknowledgment), посылаемые отправителю получателем. Отрицательные квитанции не посылаются, а для обеспечения гарантии доставки данных используется *механизм тайм-аута*. Узел назначения, получивший сегмент формирует и посылает обратно сегмент (с данными, если они есть, или без данных) с номером подтверждения, равным следующему порядковому номеру ожидаемого байта.

7. В отличие от многих других протоколов, протокол ТСР подтверждает получение *не сегментов, а байтов*. Если время ожидания подтверждения истекает, отправитель посылает сегменты еще раз.

8. Контрольная сумма (заголовок + данные + псевдозаголовки) рассчитывается в **дополнительном коде** для 16 разрядных слов.

При передаче каждого сегмента некоторого сообщения отправитель в соответствующем поле заголовка сегмента указывает номер первого байта данных, определяющий смещение этого сегмента относительно начала сообщения. Номер первого байта первого сегмента каждого сообщения формируется случайным образом *по специальному алгоритму*.

Положим, что между отправителем и получателем данных соединение установлено, тогда последовательность действий при **передаче сегментов** будет выглядеть следующим образом:

- отправитель посылает первый сегмент с данными, содержащий первые 1460 байтов передаваемого сообщения, с указанием в заголовке *номера первого байта*, формируемого по определенному алгоритму в виде некоторого случайного числа S , называемого **начальным последовательным номером** (*ISN, Initial Sequence Number*);
- если сегмент принят без ошибок, получатель отправляет сегмент с подтверждением (в заголовке флаг ACK=1), в заголовке которого в соответствующем поле указывает номер следующего ожидаемого байта ($S+1460$);
- отправитель передает следующий сегмент, содержащий следующие 1460 байтов, включая в его заголовок *номер первого байта в сегменте* ($S+1460$);
- получатель отправляет подтверждение, что он *ждет байт с номером* ($S+2920$) и т.д.;
- если сегменты придут в неправильном порядке, то получатель по номерам байтов сможет собрать их в правильной последовательности.

Существуют два **типа подтверждения**, которые могут использоваться совместно с алгоритмом скользящего окна:

- **кумулятивное** (по умолчанию) – подтверждение приема указанного байта и всех предыдущих;
- **выборочное** – подтверждение диапазона принятых байтов при большом размере окна (используется дополнительное поле заголовка TCP).

Кумулятивное подтверждение свидетельствует о том, что получен указанный байт данных и все предыдущие. В TCP такое подтверждение используется *по умолчанию*.

В то же время использование высокоскоростных каналов связи большой протяженности позволяет увеличить размер окна до 1 Гбайта. При этом, если при передаче гигабайта данных потерялся всего лишь один сегмент, который находится примерно в начале, отправителю поступит кумулятивное подтверждение о получении, например, первых 300 Мбайт, что потребует повторной передачи следующих 700 Мбайт данных, которые уже есть у получателя.

Устранение этой проблемы достигается применением **выборочного подтверждения**, при котором получатель подтверждает получение диапазона принятых байт. Он получил первые 300 Мбайт и вторые 700 Мбайт из гигабайта

и не получил всего лишь один сегмент, находящийся между ними. Отправитель вместо вторых 700 мегабайт, повторно передает всего лишь один недостающий сегмент. Выборочное подтверждение эффективно при большом размере окна ТСР, но в ТСР, как сказано выше, по умолчанию используется кумулятивное подтверждение.

Для реализации выборочного подтверждения используются *дополнительные поля заголовка ТСР – параметры*.

Слайд 27

Протокол ТСР: проблемы реализации

Несмотря на кажущуюся простоту протокола, в нем имеется ряд нюансов, которые могут привести к некоторым проблемам.

Во-первых, **сегмент или подтверждение $ACK=1$ могут быть потеряны, искажены или продублированы**. В этих случаях, как показано на рисунках, используется механизм тайм-аута, после которого отправитель повторно отправляет этот же сегмент.

Может возникнуть ситуация, когда получателем сегмент с номером первого байта 1460 был принят правильно, но подтверждение с номером ожидаемого байта 2920 не дошло до отправителя, и отправитель по истечении тайм-аута посылает тот же сегмент с тем же номером первого байта повторно. Для исключения дублирования сегментов получатель анализирует номер первого байта. Если у получателя есть сегмент с таким же номером 1460, он его отбрасывает и повторно отправляет подтверждение с указанием ожидаемого байта 2920.

Во-вторых, поскольку **сегменты при передаче по сети могут фрагментироваться**, возможна ситуация, при которой часть переданного сегмента будет принята, а остальная часть окажется потерянной.

В-третьих, **сегменты могут прибывать в узел назначения в произвольном порядке**, что может привести к ситуации, при которой байты с 2920 по 4379 уже прибыли, но подтверждение для них не может быть выслано, так как байты с 1460 по 2919 еще не получены.

В-четвертых, **сегменты могут задержаться в сети так долго, что у отправителя истечёт интервал ожидания**, и он передаст их снова. Переданный повторно сегмент может:

- пройти по другому маршруту;
- быть иначе фрагментирован;
- попасть в перегруженную сеть.

В результате для восстановления исходного сегмента потребуется достаточно сложная обработка.

В-пятых, **скорость передачи данных отправителя может превышать возможности получателя**, что приводит к возникновению перегрузок и к значительному увеличению задержек сегментов и даже к их потере. Для борьбы

с перегрузками применяются специальные методы управления трафиком для снижения негативного эффекта от перегрузок.

В-шестых, сеть, а точнее маршрутизаторы и каналы на пути передачи сегментов могут быть **перегружены**, что требует применения специальных методов *контроля и борьбы с перегрузками*.

Слайд 28

Протокол TCP: основные функции

На слайде представлена укрупненная схема организации передачи и приема данных на основе протокола TCP, включающая следующие основные функции:

- **при передаче данных:**

- *установление соединения* по запросу от приложения путем формирования и передачи адресату запроса на установление соединения;
- *прием в буфер передачи сообщения от приложения* при установленном дуплексном соединении;
- *разбиение сообщения на сегменты* определенной длины, согласованной в процессе установления соединения; в большинстве случаев размер сегмента принимается равным 1460 байтам в предположении, что на канальном уровне IP-пакеты, содержащие TCP-сегменты, будут вкладываться в Ethernet-кадры;
- *формирование TCP-заголовков* в соответствии с требованиями, предъявляемыми к качеству передачи данных со стороны приложения;
- *передача сегментов протоколу IP*, который формирует IP-пакеты, присоединяя IP-заголовки, и передает их далее протоколу канального уровня, например протоколу Ethernet.

- **при приеме данных:**

- *подтверждение соединения* на начальном этапе установления дуплексного соединения по инициативе узла-отправителя;
- *прием сегментов от протокола IP* и их запись в буфер приема;
- *анализ TCP-заголовков* и реализация функций, предписанных значениями соответствующих полей заголовка;
- *формирование сообщения* путем объединения сегментов;
- *передача сообщения приложению*.

Слайд 29

Протокол TCP: формат заголовка TCP-сегмента

Максимальный размер TCP-сегмента определяется исходя из максимального размера IP-пакета, равного 65 535 байтам, что с учетом минимально возможной длины IP-заголовка в 20 байт, составляет 65515 байт.

Первоначальный формат ТСР-сегмента, показанный на слайде, содержал в поле «Резерв» шесть неиспользуемых битов, из которых в дальнейшем 3 бита стали использоваться в качестве признаков для управления и борьбы с перегрузками.

Первые 20-байт заголовка имеют строго фиксированный формат, за которым могут находиться дополнительные поля с **параметрами**, значения которых согласовываются взаимодействующими сторонами *в процессе установления соединения*. После дополнительных полей заголовка размещается поле данных, содержащее не более 65 495 байт, которое вместе с ТСР- и IP-заголовками размером по 20 байт даст максимально допустимый размер IP-пакета в 65 535 байт.

Рассмотрим назначение фиксированных полей заголовка ТСР-сегмента.

Поля «**Порт отправителя**» (2 байта) и «**Порт получателя**» (2 байта) содержат *номера портов* приложений, между которыми установлено логическое соединение.

Поле «**Порядковый номер**» (4 байта) содержит *номер первого байта* данных в передаваемом сегменте, который определяет смещение этого сегмента относительно потока передаваемых данных, что позволяет принимающей стороне контролировать порядок сегментов и корректно восстановить всё сообщение.

В каждом ТСР-соединении поддерживается свой уникальный «порядковый номер» для отправляемых сегментов в интервале от 0 до 4 294 967 295 ($2^{32}-1$). Этот номер формируется на основе текущего 32-битового значения условного времени по определенному алгоритму.

Для исключения конфликтов, в случае совпадения порядковых номеров в сегментах из разных ТСР-соединений, *начальное значение* «порядкового номера» (ISN - Initial Sequence Number), указываемое при установлении ТСР-соединения в сегменте с флагом SYN=1, формируется следующим образом. Значение ISN увеличивается на единицу каждые 4 микросекунды и на 64 000 – каждый раз при установлении нового ТСР-соединения. Легко убедиться, что сгенерированное число будет уникальным для периода времени почти в пять часов. *Первый байт первого сегмента с данными* при установленном ТСР-соединении будет иметь номер SN=ISN+1.

После установления соединения при отправке последующих сегментов, значение порядкового номера SN увеличивается на 1 для каждого байта отправленных данных, а также для всех сегментов с флагом SYN или FIN. В последнем случае это обусловлено тем, что сегменты с флагом SYN или FIN представляют собой заголовки ТСР-сегментов без поля данных. Такой способ нумерации сегментов, а точнее первых байтов сегментов, обеспечивает принимающей системе возможность *восстановления сообщения*, объединив все сегменты в той последовательности, в какой они были сформированы отправителем.

Поле «**Подтвержденный номер**» или «Номер подтверждения» (4 байта) содержит *номер следующего ожидаемого байта* на приемной стороне при использовании получателем ТСП-сегмента в качестве положительной квитанции (флаг АСК=1), подтверждающей правильный приём всех предыдущих байтов.

Каждая сторона подсчитывает свой «порядковый номер» для переданных данных и «подтвержденный номер» для полученных данных, причем «порядковому номеру» одной стороны соответствует «подтвержденный номер» другой стороны.

Поле «**Длина заголовка**» (4 бита) задаёт длину заголовка ТСП-сегмента, измеренную в *32-битовых словах*.

Минимальный размер заголовка ТСП-сегмента составляет 20 байт, а максимальный – 60 байт. Длина заголовка определяет смещение полезных данных относительно начала сегмента.

Поле «**Резерв**» длиной 6 бит первоначально было зарезервировано. В настоящее время 3 бита в этом поле используются в качестве флагов для управления и борьбы с перегрузками, возникающими в сети.

Однобитовые **флаги** (кодовые биты) несут служебную информацию о типе сегмента и интерпретируются следующим образом:

URG=1 (от слова **urgent** – срочный) указывает на наличие *срочных данных*, что означает использование поля «Указатель на срочные данные».

ACK=1 (от слова **acknowledgement**) означает, что сегмент является *положительной квитанцией* (подтверждением) на принятый сегмент и поле «Подтвержденный номер» содержит номер ожидаемого байта. Все сегменты после стартового сегмента с SYN=1 будут иметь установленный флаг ACK=1. Если сегмент не используется как подтверждение (ACK=0), поле «Номер подтверждения» просто игнорируется.

PSH=1 (PUSH-флаг) означает *запрос на отправку данных* без ожидания заполнения буфера. При нормальной передаче данных получатель не подтверждает получение каждого сегмента, а в течение некоторого времени собирает и хранит полученные сегменты в буфере, прежде чем передать их приложению пользователя. Сегмент с флагом PUSH=1 предписывает получателю немедленно передать приложению пользователя все полученные сегменты из буфера и послать отправителю подтверждение.

RST=1 (от слова **reset** – сброс) используется для *сброса состояния соединения* при обнаружении проблем, ошибок и нестандартных ситуаций, а также для отказа от неверного сегмента или от попытки создать соединение.

SYN=1 (от слова **syncing** – синхронизация) используется для *установки соединения*, при этом если ACK=0, то это означает, что поле подтверждения не используется;

FIN=1 (от слова **finish** – финиш) используется для *разрыва соединения*.

Поле «**Размер окна**» (2 байта) определяет, сколько байт может быть послано после байта, получившего подтверждение.

Поле **«Контрольная сумма»** (2 байта) содержит контрольную сумму, которая охватывает заголовок, данные и **псевдозаголовок**.

Поле **«Указатель на срочные данные»** (2 байта) при установленном флаге $URG=1$ содержит численное значение смещения в байтах от «Порядкового номера». Смещение указывает на последний байт срочных данных в сегменте, которые необходимо немедленно принять, даже если буфер приема переполнен. Например, если сегмент содержит 1000 байт срочных данных, после которых находятся 460 байт обычных данных, будет установлен флаг $URG=1$, а «Указатель на срочные данные» будет иметь значение 1000. Таким образом в протоколе TCP реализуются *прерывающие сообщения*. Содержимым срочных данных занимается прикладной уровень. Протокол TCP лишь обеспечивает их доставку и не интересуется причиной прерывания.

TCP предоставляет процессу возможность приоритетной отправки данных с использованием флага URG . Процесс, которому необходимо отправить срочные данные, отправляет их протоколу TCP с флагом $URG=1$.

Протокол TCP создает сегмент с флагом $URG=1$, и в поле «Указатель на срочные данные» записывает значение смещения, указывающее на последний байт срочных данных в сегменте. Например, если в поле данных сегмента находится 1000 байт срочных данных, за которыми следуют обычные данные, будет установлен флаг $URG=1$, а в поле «Указатель на срочные данные» будет занесено значение 1000.

При получении сегмента с флагом $URG=1$ на принимающей стороне по значению в поле «Указатель на срочные данные» определяется, какие данные в сегменте являются срочными, которые затем пересылаются процессу с указанием «срочные». Остальные данные в сегменте обрабатываются обычным способом.

Поле **«Параметры»** имеет переменную длину и содержит параметры, значения которых согласовываются в процессе установления соединения. При передаче данных после установления соединения это поле обычно отсутствует.

Примерами приложений, использующих протокол TCP для передачи данных, являются FTP, TFTP, DNS, POP3, IMAP, TELNET.

Слайд 30

Протокол TCP: флаги для управления перегрузкой

В последней версии протокола TCP для управления перегрузкой в поле «Резерв» были добавлены еще 3 однокбитовых флага:

- ECE (ECN-Echo) выполняет двойную роль в зависимости от значения флага SYN:
 - при установлении соединения ($SYN=1$) указывает, что отправитель поддерживает ECN;
 - при передаче данных ($SYN=0$) указывает отправителю о перегрузке (или надвигающейся перегрузке) в сети;

- CWR (Congestion Window Reduced) – «окно перегрузки уменьшено» – устанавливается отправителем для подтверждения получения сегмента с флажком ECE от узла-получателя и включения отправителем механизма уменьшения перегрузки (Congestion Control);
- NS (Nonce Sum) – «одноразовая сумма» – для улучшения работы механизма *явного уведомления о перегрузке* (Explicit Congestion Notification, ECN).

В качестве дополнительных параметров в заголовке сегмента TCP могут указываться:

1) максимальный размер сегмента MSS (Maximum Segment Size), который устанавливается и согласовывается отправителем и получателем при установке соединения (для Ethernet - 1460 байт);

2) масштаб окна, позволяющий увеличить размер окна до 1 ГБ вместо 65535 байт в случае *высокоскоростных каналов* связи между отправителем и получателем;

3) выборочное подтверждение диапазона принятых байт, а не всех данных, если потерян всего лишь один сегмент или небольшая часть в большом потоке данных.

Слайд 31

Протокол TCP: вычисление контрольной суммы

Алгоритм вычисления контрольной суммы выглядит следующим образом.

Перед началом вычисления контрольной суммы отправителем сегмента значение этого поля устанавливается равным нулю.

Если поле данных содержит нечётное число байтов, то оно дополняется нулевым байтом, который используется при подсчёте контрольной суммы, но не вставляется в сегмент для передачи в сети. Необходимость такого добавления обусловлена тем, что TCP-сегмент, включающий *заголовок, данные и псевдозаголовок*, рассматривается как совокупность 16-разрядных двоичных чисел, которые складываются. Если длина N суммы превышает 16 разрядов, то $(N-16)$ старших разрядов суммы рассматриваются как младшие разряды 16-разрядного двоичного числа (путем добавления нулей в старшие разряды), которое добавляется к этой сумме. Затем окончательно полученная сумма инвертируется (переводится в дополнительный код) и заносится в поле «Контрольная сумма».

Получатель сегмента аналогичным образом подсчитывает контрольную сумму для всего сегмента, включая поле «Контрольная сумма». Очевидно, что полученный таким образом результат должен быть равен 0.

Отметим, что *дополнительный нулевой байт и псевдозаголовок не передаются по сети*.

Слайд 32

Протокол TCP: установление и разрыв соединения

Для передачи данных протокол TCP использует соединение, которое должно быть установлено до начала передачи и разорвано после завершения передачи данных.

В соответствии с протоколом TCP передача данных организуется в 3 этапа:

1) **установка соединения** - «тройное рукопожатие»:

- *отправитель* (клиент) посылает TCP-сегмент без данных с флагом SYN=1, означающем запрос на соединение, и случайным числом «ISN1» в поле «Порядковый номер» (Sequence Number);
- *получатель* (сервер) при наличии возможности выделяет системные ресурсы (буферы, таймеры и т.п.) и посылает отправителю TCP-сегмент с флагами SYN=1 и ACK=1, означающими, соответственно, запрос на соединение и подтверждение возможности организации соединения, при этом поле «Порядковый номер» будет содержать другое случайное число «ISN2», а поле «Подтвержденный номер» будет содержать значение «ISN1+1», что соответствует следующему порядковому номеру, который получатель ожидает получить от отправителя (*в случае невозможности по каким-то причинам установить соединение получатель посылает отправителю сегмент с флагом RST=1 и отправитель прекращает попытки соединиться*);
- при получении подтверждения ACK=1 на установление соединения *отправитель* посылает получателю ответное подтверждение (ACK=1), а в поле «Подтвержденный номер» указывает значение «ISN2+1», что соответствует следующему порядковому номеру, который отправитель ожидает получить от получателя, после чего дуплексное TCP-соединение считается установленным;

2) **передача данных**: отправитель может передавать данные, впрочем, как и получатель, который может передавать TCP-сегменты с данными, предназначенными для отправителя, при этом *все сегменты в обязательном порядке будут содержать установленный флаг ACK=1*, а другие флаги, такие как PSH или URG, могут быть установлены или не установлены;

3) **разрыв соединения** выполняется с использованием флагов FIN=1 (при этом сохраняется возможность односторонней передачи данных) или RST=1, означающее принудительный разрыв и прекращение передачи данных.

При установке TCP-соединения необходимо:

- убедиться, что отправитель и получатель готовы передавать данные друг другу;
- договориться о нумерации потоков байт, поскольку нельзя постоянно нумеровать данные в первом сегменте с нуля; номера первого байта

каждый раз выбираются по определенному алгоритму, причем отправитель и получатель договариваются о том, какое начальное значение они будут использовать для нумерации потока байт в процессе установления ТСП-соединения; в общем случае можно считать, что они выбираются случайным образом;

- договориться о параметрах соединения (максимальный размер сегмента, начальное значение первого байта в первом сегменте и т.п.).

Протокол ТСП предусматривает три варианта **разрыва соединения**:

- *корректный односторонний разрыв* соединения путем послыки сегмента с флагом FIN=1 только с одной стороны;
- *корректный одновременный (двухсторонний) разрыв* соединения путем послыки сегментов с флагом FIN=1 с обеих сторон;
- *принудительный разрыв* соединения при возникновении нестандартной или критической ситуации путем послыки сегмента с флагом RST=1.

Корректный разрыв соединения может инициировать любая из двух сторон, участвующих в ТСП-соединении, если у нее закончились данные для передачи, путем послыки другой стороне сегмента с флагом FIN=1 и получения в ответ подтверждения с флагом ACK=1. При этом соединение оказывается разорванным только с одной стороны, но другая сторона может продолжать передачу данных.

Когда и у другой стороны закончились данные для передачи, она также передает сегмент с флагом FIN=1, а в ответ получает подтверждение – сегмент с флагом ACK=1. Таким образом ТСП-соединение оказывается полностью разорванным.

В критической ситуации при возникновении ошибок в приложении или проблем с оборудованием для **принудительного разрыва** соединения посылается сегмент с флагом RST=1. При этом разрыв соединения выполняется с обеих сторон. Иногда такая процедура используется для быстрого закрытия соединения.

Слайд 33

Протокол ТСП: перегрузка и управление потоком

Специфическими особенностями протокола ТСП по сравнению с протоколом UDP при передаче сообщения по установленному ТСП-соединению являются следующие:

- сегменты сообщения в рамках установленного *дуплексного соединения* передаются в виде *упорядоченной последовательности*;
- при возникновении ошибки в переданном сегменте или потере сегмента осуществляется *повторная передача* сегмента;
- за счет применения метода управления трафиком на основе «скользящего окна» осуществляется контроль передаваемых данных;

- при возникновении перегрузок реализуются специальные методы борьбы с перегрузкой путем ограничения скорости передачи данных отправителем вплоть до полной остановки передачи путем указания получателем нулевого размера окна.

Для передачи данных по ТСП-соединению в компьютерах отправителя и получателя формируются 3 буфера: отправления, копий и приёма. Сегменты после отправки сохраняются в буфере копий до момента получения отправителем подтверждения от получателя.

В качестве признака (сигнала) перегрузки в сети может служить:

- потеря сегментов;
- увеличенная задержка сегментов;
- сигнал от маршрутизатора о перегрузке.

Если скорость, с которой узел-отправитель посылает сегменты, превышает возможности узла-получателя, возникает перегрузка узла-получателя, при которой буфер последнего оказывается заполненным до предела, после которого узел-получатель начинает отбрасывать поступающие сегменты. Узел-отправитель, не получив подтверждения в течение тайм-аута, повторно посылает потерянный сегмент, что только увеличивает перегрузку. Следует иметь в виду, что перегрузка может возникнуть не только в узле-получателе, но и в промежуточных узлах, например, в маршрутизаторах. Если узел-отправитель получает сигнал о перегрузке, он может принять **меры по снижению нагрузки** в сети за счет:

- увеличения *тайм-аута*;
- изменения (адаптивного уменьшения) *размера окна*, диктуемого узлом-получателем сегментов, вплоть до полной приостановки передачи.

В последнем случае размер окна устанавливается как наименьшее значение из размера **окна управления потоком**, диктуемого узлом-получателем путем явного указания в ответном сегменте, и размера **окна перегрузки**, рассчитанного и уменьшенного при возникновении перегрузки с использованием какого-либо метода управления и борьбы с перегрузками.

ТСП-сегмент, *вложенный в IP-пакет*, из-за перегрузки сети или непредвиденной ситуации в сети может быть *потерян* или *доставлен не по порядку*, поскольку протокол IP не гарантирует надежную доставку. Обеспечение надежной доставки возложено на протокол ТСП, который:

- *обнаруживает ошибки*;
- реализует *повторную передачу* потерянных или искаженных сегментов;
- решает проблемы, связанные с *перегрузкой* сети.

При успешной передаче сообщения протокол ТСП в узле-получателе восстанавливает из полученной последовательности сегментов сообщение и передает его принимающему приложению.

Следует отметить, что протокол TCP ориентирован на *надежную*, а не на *своевременную доставку*. Поскольку в случае повторной передачи потерянных сегментов задержка доставки сообщений может достигать нескольких секунд, протокол TCP *не используется для приложений реального времени*, например для передачи голоса по IP. Обычно для таких приложений используется транспортный протокол реального времени (RTP), работающий поверх протокола UDP.

Протокол TCP используется многими интернет-приложениями, включая World Wide Web (WWW), электронную почту, протокол передачи файлов, потоковое мультимедиа.

Слайд 34

Протокол TCP: методы управления перегрузкой

Размер окна, который устанавливается отправителем при передаче сегментов получателю определяется как минимальное значение из двух значений:

- размера окна, объявленного получателем сегментов в зависимости от заполненности его входного буфера (**окно управления потоком**) и передаваемого отправителю с каждым подтверждением;
- размера окна, рассчитанный (уменьшенный) при возникновении перегрузки в сети (**окно перегрузки**).

В первом случае получатель при отправке TCP-сегмента с подтверждением (ACK=1) указывает в заголовке *размер окна* – количество байт, которые он может принять.

Второй случай связан с перегрузкой в сети, при возникновении которой происходит переполнение буферной памяти маршрутизаторов, и они начинают отбрасывать поступающие сегменты. Отправители, не получая подтверждения (положительной квитанции) в течение тайм-аута, повторно посылают отброшенные сегменты, что приводит к ещё большей перегрузке. При этом возникает ситуация, при которой каналы сети заняты передачей, но сегменты от отправителя до получателя практически не доходят. Чем дольше работает сеть, тем больше перегрузка и, как следствие, больше теряется сегментов, что в конечном счете приводит к полной деградации сети и невозможности передачи сегментов. Такая ситуация называется **коллапсом перегрузки** и впервые произошла в сети Интернет в 1986 году. Стало очевидным, что для решения этой проблемы при формировании размера окна необходимо учитывать загрузку сети. Это означает, что размер окна должен динамически изменяться в зависимости от загрузки сети. Таким образом, размер окна на стороне отправителя должен определяться не только с учетом того, сколько данных может принять получатель, но и с учетом загрузки сети.

Совместное использование окна управления потоком и окна перегрузки в протоколе TCP позволяет решать более общую задачу управления скоростью передачи данных за счет выбора **оптимального размера окна**.

Действительно, при *маленьком размере окна* в сеть будет посылаться небольшое количество сегментов и скорость передачи окажется маленькой.

С другой стороны, при *большом размере окна* в сеть будет посылаться большое количество сегментов и сеть может оказаться перегруженной. В этом случае маршрутизаторы начнут отбрасывать сегменты, а отправители будут повторно отправлять их в сеть. При этом эффективная (полезная) скорость передачи данных опять окажется низкой. Таким образом, можно говорить о существовании *оптимального размера окна*, при котором эффективная скорость передачи данных будет **максимальной**.

Для выбора и адаптивного изменения размера окна с учетом загруженности сети **разработаны методы управления перегрузкой сети (маршрутизаторов)**:

- **AIMD** (*Additive Increase / Multiplicative Decrease*) - метод аддитивного увеличения, мультипликативного уменьшения (на медленных каналах);
- «медленный старт»;
- «медленный старт» с AIMD в протоколе TCP.

AIMD - метод аддитивного увеличения / мультипликативного уменьшения

В TCP для определения размера окна перегрузки используется метод AIMD (*Additive Increase / Multiplicative Decrease* – аддитивное увеличение / мультипликативное уменьшение), суть которого в том, что при получении подтверждения к текущему размеру окна прибавляется значение, обычно равное длине одного сегмента TCP. При возникновении перегрузки размер окна уменьшается, например, в два раза:

$$w(t+1) == \begin{cases} w(t) + a, & \text{если нет перегрузки;} \\ w(t) * b, & \text{если есть перегрузка,} \end{cases}$$

где $t = 0, 1, 2, \dots$; $w(0) = 0$; $a = \text{MSS}$ – максимальный размер сегмента (по умолчанию $\text{MSS}=536$; $\text{MSS}=1460$ для сети Ethernet); $b = 0,5$.

На графике показан принцип реализации метода AIMD.

После каждого подтверждения от получателя размер окна увеличивается на длину одного сегмента, т.е. происходит аддитивное увеличение. При возникновении перегрузки размер окна уменьшается в два раза (мультипликативное уменьшение). Затем размер окна снова увеличивается на длину одного сегмента после каждого подтверждения и так до появления очередной перегрузки.

Медленный старт

Метод аддитивного увеличения мультипликативного уменьшения AIMD хорошо работал на медленных каналах связи, но он оказался неэффективным для современных высокоскоростных и надежных каналов, поскольку размер окна перегрузки при использовании AIMD увеличивается медленно.

Для решения этой проблемы был разработан метод управления размером окна, получивший название «медленный старт»:

$$w(t+1) = \begin{cases} 2w(t), & \text{если нет перегрузки;} \\ 0, & \text{если есть перегрузка,} \end{cases}$$

где $w(0) = a$; $a = \text{MSS}$.

В этом методе на каждое полученное подтверждение размер окна увеличивается вдвое по сравнению с предыдущим значением. Благодаря этому размер окна увеличивается экспоненциально: сначала его размер равен длине одного сегмента, а затем после получения каждого подтверждения размер увеличивается вдвое. Можно отметить, что, несмотря на название «медленный старт», размер окна увеличивается гораздо быстрее, чем в методе AIMD. С другой стороны, если произошла потеря сегмента, то *размер окна уменьшается до нуля*, т.е. «медленный старт» быстро разгоняется, но при возникновении перегрузки сразу уменьшается до нуля.

Медленный старт и AIMD в TCP

В протоколе TCP используется комбинация методов «медленный старт» и «аддитивное увеличение / мультипликативное уменьшение». Сначала используется медленный старт, для того чтобы быстро заполнить доступную пропускную способность канала. После того, как размер окна достиг определенного значения, называемого **«порогом медленного старта»**, происходит переход на метод «аддитивное увеличение / мультипликативное уменьшение» (метод AIMD), при этом размер окна увеличивается медленно, а при возникновении перегрузки, размер окна будет уменьшен в два раза, а не до нуля.

«Порог медленного старта» определяется следующим образом. Сначала запускается медленный старт, который идет до тех пор, пока не поступит сигнал о перегрузке. После этого размер окна, при котором возникла перегрузка, уменьшается в два раза, и полученное значение в дальнейшем используется в качестве *порога медленного старта*. Это позволяет обеспечить достаточный запас размера окна, для того чтобы не произошла перегрузка. Другими словами, если загрузка сети сохраняется, вероятность возникновения перегрузки в процессе аддитивного увеличения незначительна. В то же время, если перегрузка сети всё-таки произошла, реализуется мультипликативное уменьшение размера окна в 2 раза.

Слайд 35

Протокол TCP: сигналы о перегрузке

Для реализации рассмотренных методов борьбы с перегрузкой отправитель должен каким-то образом получить информацию о возникновении перегрузки в сети. Это – не простая задача, поскольку перегрузка может происходить не в той подсети, к которой подключен отправитель, а в подсети, которая находится достаточно далеко от отправителя и получателя.

В качестве признака (сигнала) перегрузки могут использоваться следующие показатели:

- потери сегментов;
- задержки сегментов;
- сигнал о перегрузке от маршрутизатора.

Потери сегментов. Современные каналы связи в компьютерных сетях достаточно хорошего качества, поэтому принято считать, что потери сегментов происходят не из-за ошибки канала, а из-за перегрузки сети и поэтому для уменьшения перегрузки необходимо уменьшать размер окна для передаваемых сегментов.

К недостаткам использования в качестве признака перегрузки показателя «потери сегментов» относятся:

- *перегрузка не предотвращается*, поскольку информация для борьбы с перегрузкой поступает отправителю тогда, когда перегрузка уже произошла;
- когда в маршрутизаторе из-за перегрузки заканчивается место в буфере, отбрасываются сегменты всех отправителей, что создает ситуацию, при которой все отправители после уменьшения размера окна начинают практически одновременно передавать сегменты, в результате чего в маршрутизатор поступает много сегментов, что приводит к перегрузке.

Для решения этих проблем используются другие показатели перегрузки.

Задержки сегментов. В этом случае отправитель измеряет время RTT (Round Trip Time) передачи сегмента от отправителя до получателя и обратно. Признаком начала перегрузки в сети может служить существенное увеличение RTT по сравнению с некоторым средним значением, полученным по результатам предыдущих измерений времени RTT. В этом случае отправитель полагает, что в сети начинается перегрузка, и уменьшает размер окна перегрузки. Таким образом перегрузка может быть обнаружена до того, как она произошла.

К недостаткам использования в качестве признака перегрузки показателя «задержка сегмента» относятся следующие:

- задержки сегментов могут быть обусловлены причинами, не связанными с перегрузкой сети, и, следовательно являются не такими объективными показателями, как потери сегментов;
- несправедливое распределение пропускной способности каналов связи, когда компьютер, обнаруживший перегрузку по значению RTT, уменьшает размер окна перегрузки, другие компьютеры, использующие в качестве показателя перегрузки «потерю сегмента», увеличивают размер окна до тех пор, пока перегрузка не произойдет.

Очевидно, что разумным решением является совместное использование двух показателей: «задержки сегмента» и «потери сегмента». Такой подход реализован в протоколе Compound TCP фирмы Microsoft.

Сигнал о перегрузке от маршрутизатора. Ещё один способ обнаружения перегрузки – сигнал от маршрутизатора. Для этого маршрутизаторы должны поддерживать отправку сигналов о перегрузке. При этом возможны два варианта реализации этого способа:

- маршрутизатор начинает отбрасывать пакеты заранее до того, как буфер заполнится, в результате чего отправитель уменьшит окно перегрузки до её возникновения (неявный сигнал); этот способ реализован, например, в технологии Random Early Detection (RED);
- маршрутизатор реализует явную передачу сигнала о перегрузке отправителю, например, как в технологии Explicit Congestion Notification, встроенной в протокол TCP; для этого в заголовок IP-пакета включены два бита, которые могут принимать два значения: 00 – нет перегрузки и 11 – при возникновении перегрузки.

Слайд 36

ECN – технология явного уведомления о перегрузке

Рассмотрим реализованную совместно в протоколах IP и TCP технологию «явного уведомления о перегрузке» ECN (Explicit Congestion Notification).

Положим, что отправитель передает сегмент в сеть, который доходит до маршрутизатора, находящегося в состоянии близком к перегрузке – буфер заполнен, но не полностью. Для того чтобы предупредить отправителя о приближающейся перегрузке, маршрутизатор устанавливает в заголовке **IP-пакета**, внутри которого находится передаваемый сегмент протокола TCP, специальный двухбитовый флаг ECN (ECN=11), указывающий на перегрузку в сети.

Сегмент передается далее по сети и достигает получателя. Получатель обнаруживает в заголовке IP-пакета флаг ECN=11, свидетельствующий о перегрузке. Для того чтобы о перегрузке узнал отправитель, получатель, формируя сегмент-подтверждение, устанавливает в заголовке TCP-сегмента флаг ECE=1.

Отправитель при получении подтверждения о доставке сегмента видит, что флаг ECE=1, сигнализирующий о перегрузке, установлен. Это означает, что нужно уменьшить размер окна перегрузки. Отправитель уменьшает размер окна перегрузки и в очередном посылаемом сегменте устанавливает флаг CWR=1, подтверждая получателю уменьшение окна.

Слайд 37

Сравнение протоколов TCP и UDP

Сравнение транспортных протоколов TCP и UDP выполняется на основе следующих показателей:

- размер заголовка блока данных;
- название блока данных (PDU);

- необходимость установления логического соединения;
- действия при обнаружении ошибок (контроль ошибок);
- соблюдение последовательность доставки;
- упорядоченность доставки;
- наличие контроля и управления перегрузками;
- наличие накладных расходов;
- приложения, использующие протокол.

Слайд 38

Многопутевой TCP - Multipath TCP (MPTCP)

MPTCP (*Multipath TCP*)– технология передачи данных по протоколу TCP одновременно по нескольким каналам связи с целью увеличения пропускной способности и надежности за счет максимальной загрузки ресурсов компьютерной сети и увеличения избыточности (спецификация RFC 6824, 2013 год).

При многопутевом TCP соединение устанавливается между двумя хостами, а не между двумя интерфейсами (IP-адресами), как в TCP, в котором соединение идентифицируется двумя сокетами, что позволяет создать только одно TCP-соединение.

Многопутевой TCP позволяет соединению использовать несколько путей одновременно. В MPTCP по каждому пути создается TCP-соединение, называемое *подпоток* (sub-flow).

Основными операциями протокола MPTCP являются:

- **добавление / удаление пути** (например, если соединение потеряно из-за перегрузки);
- **обеспечение совместимости с устаревшим оборудованием** TCP (например, с брандмауэрами, которые могут автоматически отклонять TCP-соединения, если порядковый номер не является последовательным);
- **определение стратегии справедливого контроля перегрузки** между разными соединениями и разными хостами (особенно с теми, которые не поддерживают MPTCP).

Реализация MPTCP предполагает использование новых специальных механизмов, таких как:

- 1) **система подпотока**, используемая для сбора нескольких стандартных TCP-соединений (путей от одного хоста к другому) и обеспечивающая **добавление** или **удаление подпотоков** для создания нескольких TCP-путей в рамках одного логического соединения;
- 2) **опция MPTCP DSS** (*Data Sequence Signal* – сигнал последовательности данных), содержащая **порядковый номер данных** и **номер подтверждения**,

что позволяет получать данные из нескольких подпотоков *в исходном порядке*;

- 3) **модифицированный протокол повторной передачи**, обеспечивающий контроль и управление перегрузкой, а также *повышенную надежность*, не проявляя при этом несправедливости к TCP-соединениям с одним путем, которые могут конкурировать на каком-то пути.

Слайд 39

3.5. Коммуникационный протокол IPv4

Протокол IP специфицирует три основных элемента:

- блок данных – **пакет IP**, с которым работает протокол;
- механизмы передачи пакетов;
- способы обработки конфликтных ситуаций.

Пакет IP состоит из заголовка и блока данных, содержащего UDP-дейтаграмму, TCP-сегмент, сообщения, формируемые другими протоколами (ICMP, RIP, OSPF и т.д.).

В настоящее время в сети Интернет могут циркулировать IP-пакеты двух версий:

- IP-пакеты версии 4 (IPv4);
- IP-пакеты версии 6 (IPv6).

Протокол IP *обрабатывает и интерпретирует только поля заголовка*.

Формат заголовка пакета IPv4 показан на слайде.

Рассмотрим назначения полей заголовка.

«**Номер версии**» (4 бита) – используется для указания версии протокола IP, который должен обрабатывать данный пакет. В настоящее время осуществляется постепенный переход от версии 4 к версии 6, и большинство узлов могут обрабатывать пакеты обеих версий. Если это поле содержит значение, отличное от указанных версий протокола, пакет *уничтожается*.

«**Длина заголовка**» (4 бита) – задает значение длины заголовка пакета, измеренной в 32-битовых (4-байтовых) словах. Минимальное значение длины (при отсутствии необязательных полей «Параметры» и «Наполнение») равно 5, что соответствует заголовку длиной 20 байт. Максимальное значение этого 4-битового поля равно 15, что соответствует заголовку длиной 60 байт. Следовательно, максимальный размер необязательных полей «Параметры» и «Наполнение» равен 40 байтам.

«**Тип сервиса**» (Type of Service, ToS) – первоначально это 8-битовое поле было предназначено для оптимизации транспортной службы и содержало:

- 3-битовое поле «Приоритет» – 8 значений: от 0 (нормальный приоритет) до 7 (сетевое управление – высший приоритет);
- биты D, T, R – означали требование:

- D=1 (Delay – *задержка*) – малой задержки при передаче пакета;
- T=1 (Throughput – *пропускная способность*) – высокой пропускной способности;
- R=1 (Reliability – *надежность, достоверность*) – высокой надежности.

Стандарты, принятые в конце 90-х годов прошлого века, дали новое название этому полю – байт **дифференцированного обслуживания** (*Differentiated Service, DS*) или **DS-байт** – и переопределили назначение его битов. Первые 6 битов получили название **DSCP** (*Differentiated Services Code Point* – точка кода дифференцированных услуг) и используются для разделения трафика на классы обслуживания, предоставляя больший приоритет, например, чувствительному к задержкам трафику – голосовому (VoIP) или видео.

Кроме того, два ранее резервных бита были задействованы для *явного уведомления о перегрузке маршрутизаторов* на пути передачи пакетов и получили название **ECN** (*Explicit Congestion Notification*): 00 – нет перегрузки и 11 – маршрутизатор перегружен.

«**Общая длина**» (16 бит) – задает длину пакета, включая заголовок и данные, измеренную в байтах. Теоретически общая длина пакета IP может достигать 65 535 байт, однако в большинстве сетей столь большие пакеты не используются.

Протокол IP должен обеспечивать межсетевое взаимодействие между разными сетями, различающимися, в том числе, ограничением на *максимальную длину кадра*, разрешенным в той или иной физической сети (Maximum Transfer Unit, MTU). Поэтому протокол IP вынужден решать задачу, более свойственную транспортному протоколу, – разбивку больших пакетов на малые и наоборот – их сборку. Это требуется делать в тех случаях, когда на вход некоторой физической сети поступает пакет, превосходящий по длине MTU для данной сети. Такая операция называется **фрагментированием** (fragmentation) и осуществляется следующим образом.

Блок данных большого исходного пакета разделяется на **фрагменты** длиной MTU для физической сети, в которую направляются фрагменты. При этом фрагменты упаковываются в пакеты, заголовки которых похожи на заголовок исходного пакета.

В стандартах TCP/IP предусматривается, что все узлы должны принимать пакеты длиной не менее 576 байт, независимо от того, являются они фрагментами или целыми пакетами.

Следующие три поля заголовка пакета указывают на то, что данные пакеты являются фрагментами одного большого пакета.

«**Идентификатор пакета**» (16 бит) – общий для всех фрагментов идентификатор, указывающий на принадлежность фрагмента к одному большому пакету.

«**Флаги**» (3 бита) – содержат признаки (биты), связанные с фрагментацией:

- **DF** (Do not Fragment – не фрагментировать) – значение, равное 1, запрещает маршрутизатору фрагментировать пакет;
- **MF** (More Fragments – больше фрагментов) – значение, равное 1, означает, что фрагмент является промежуточным;
- один бит зарезервирован.

«**Смещение фрагмента**» (13 бит) – смещение в байтах поля данных этого фрагмента относительно начала поля данных исходного не фрагментированного пакета. Смещение используется при сборке фрагментов в пакет и должно быть кратно 8 байтам.

«**Время жизни**» (Time To Live, TTL) – 8-битовое поле, содержащее время, измеряемое в секундах, в течение которого пакет может существовать в сети. Хосты и маршрутизаторы, обрабатывающие данный пакет, уменьшают значение этого поля в период обработки и хранения пакета как минимум на 1 плюс время ожидания в очереди. Однако на практике в каждом маршрутизаторе обычно из этого времени просто вычитается 1. Таким образом, время жизни фактически измеряется количеством маршрутизаторов, через которые проходит пакет. Когда время жизни истекает, пакет уничтожается. При этом источник сообщения уведомляется о потере пакета. Наличие конечного времени жизни пакета, равное 255 (8 двоичных разрядов), обеспечивает, в частности, защиту от таких нежелательных событий, как передача пакета по циклическому маршруту, перегрузка сетей.

«**Протокол**» (8 бит) – указывает протокол вышележащего уровня, которому предназначена информация, содержащаяся в поле данных пакета IP. Например, значение 6 соответствует протоколу TCP, значение 17 – протоколу UDP, значение 1 – протоколу ICMP.

«**Контрольная сумма заголовка**» (16 бит) – используется для контроля целостности только заголовка пакета IP и вычисляется как сумма всех 16-битовых полуслов заголовка в дополнительном коде, преобразованная также в дополнительный код. Таким образом, вычисляемая получателем контрольная сумма заголовка вместе с этим полем должна быть равна нулю. Поскольку некоторые поля заголовка могут изменять свои значения в процессе передачи пакета по сети, контрольная сумма вычисляется и проверяется в каждом маршрутизаторе и в конечном узле.

«**IP-адрес источника**» (32 бита) – IP-адрес отправителя пакета.

«**IP-адрес назначения**» (32 бита) – IP-адрес получателя пакета.

«**Параметры**» – необязательное поле переменной длины, применяемое для указания параметров, используемых обычно при отладке сети и связанных, например, с режимами безопасности или маршрутизации.

«**Наполнение**» – поле переменной длины, необходимое для дополнения заголовка пакета до целого числа 32-битовых слов.

Слайд 40

3.6. Протоколы маршрутизации

Методы маршрутизации

Все методы маршрутизации, применяемые в маршрутизаторах, можно разбить на две группы:

- 1) методы *статической (фиксированной)* маршрутизации;
- 2) методы *динамической (адаптивной)* маршрутизации.

Статическая маршрутизация означает, что пакеты передаются по определенному пути, установленному администратором и не изменяемому в течение длительного времени.

Статическая маршрутизация применяется в небольших мало изменяющихся сетях, благодаря таким *достоинствам* как:

- низкие требования к маршрутизатору;
- повышенная безопасность сети.

В то же время статической маршрутизации присущи следующие *недостатки*, существенно ограничивающие её применение:

- высокая трудоемкость эксплуатации (сетевые администраторы должны задавать и модифицировать маршруты вручную);
- нет адаптации к изменениям топологии сети.
-

Динамическая маршрутизация – распределенная маршрутизация, позволяющая автоматически изменять маршрут следования пакетов при отказах или перегрузках каналов связи.

Для автоматического построения и модификации маршрутных таблиц используются протоколы:

- *внутренней маршрутизации* – IGP (Interior Gateway Protocol), например RIP, OSPF, IS-IS, ES-IS;
- *внешней маршрутизации* – EGP (Exterior Gateway Protocol), на смену которому пришел протокол BGP (Border Gateway Protocol), используемый в настоящее время в сети Internet.

С использованием **протоколов внутренней маршрутизации** маршрутные таблицы строятся в пределах так называемой **автономной системы** (autonomous system), представляющей собой совокупность сетей с единым административным подчинением.

Для обмена маршрутной информацией между автономными системами (АС) чаще всего применяется **протокол внешней маршрутизации** EGP, разработанный для сети Internet. Этот протокол назван так потому, что внешний маршрутизатор, как правило, размещается на периферии автономной системы.

Задачи протокола EGP:

- сбор информации о доступности всех сетей своей автономной системы;
- передача собранной информации внешним маршрутизаторам других автономных систем.

С учетом опыта применения протокола EGP был разработан протокол BGP, основанный на использовании надежного транспортного протокола TCP, который по сравнению с EGP:

- обеспечивает более быструю стабилизацию оптимальных маршрутов;
- меньше загружает сеть служебной информацией, в частности, за счет передачи при изменении сети информации, относящейся только к этому изменению.

Основным сообщением протокола BGP является **сообщение UPDATE** (сообщить), с помощью которого маршрутизатор сообщает соседним маршрутизаторам о появлении или исчезновении сети в своей автономной системе, т.е. это **триггерное сообщение**, которое формируется автоматически при подключении или отключении АС и каналов связи. Одно такое сообщение содержит информацию о появлении одного нового маршрута или об аннулировании несколько маршрутов. В протоколе BGP маршрут задается в виде последовательности автономных систем на пути к определенной сети, причем нумерация АС исключает заикливание UPDATE-сообщений.

Например, при появлении в автономной системе АС5 новой сети с адресом **210.5.6.0/24** маршрутизатор EGP3 в процессе сеанса сформирует и направит маршрутизатору EGP1 следующее UPDATE-сообщение:

АС5_195.2.3.1_210.5.6.0/24,

где **195.2.3.1** – адрес интерфейса маршрутизатора EGP3, на который следует присылать пакеты, направляемые в сеть **210.5.6.0/24**.

В свою очередь, маршрутизатор EGP1 распространит полученную информацию в своей автономной системе АС1, используя протокол внутренней маршрутизации IGP.

Маршрутизатор может обмениваться маршрутной информацией по протоколу BGP с другим маршрутизатором только в том случае, если администратор явно указал их связь при конфигурировании.

В качестве транспорта сообщения BGP используют *протокол TCP*.

К протоколам внутренней маршрутизации на основе алгоритма LSA относятся следующие протоколы сетевого уровня:

- IS-IS (*Intermediate System–Intermediate System*) – транзитная система–транзитная система;
- ES-IS (*End System–Intermediate System*) – оконечная система–транзитная система";
- RIP (*Routing Information Protocol*);
- OSPF (*Open Shortest Path First*).

Слайд 41

Протоколы внутренней маршрутизации

Протоколы маршрутизации управляют динамическим обменом информацией о маршрутах между всеми маршрутизаторами сети, реализуются

программно в маршрутизаторе, создавая таблицы маршрутизации, отображающие организацию всей сети.

Протоколы внутренней маршрутизации, как правило, основаны на алгоритмах обмена:

- таблицами "вектор-длина" – DVA (Distance Vector Algorithm) – протоколы типа «distance vector»;
- информацией о состоянии каналов – LSA (Link-State Algorithm) – протоколы типа «link state».

DVA - алгоритм обмена информацией о доступных сетях и расстояниях до них путём *периодической* рассылки маршрутизаторами широковещательных пакетов.

К протоколам типа DVA относится один из самых первых протоколов **RIP (Routing Information Protocol)**, который первоначально широко применялся в сети Интернет.

В соответствии с этим протоколом каждый маршрутизатор периодически (даже если в сети не происходит изменений) посылает соседним маршрутизаторам широковещательные пакеты (RIP-пакеты) со своей таблицей маршрутизации, в соответствии с которой маршрутизаторы обновляют свои таблицы маршрутизации.

Формат RIP-пакета включает следующие поля:

- **Command**, определяющий тип пакета и принимающий значения: 1 – request; 2 – response;
- **Version – версия протокола:** 1 или 2 (версия 2 содержит дополнительную маршрутную информацию и обеспечивает более высокий уровень безопасности);
- **Routing Domain** – 0 (не используется);
- **RIP Entry** – записи маршрутной информации, содержащие в том числе:
 - тип адреса (2 для IP-адреса);
 - IP-адрес назначения;
 - *маску подсети* (в RIP версии 2)
 - метрику.

При *инициализации* маршрутизатора в таблицу маршрутизации записываются:

- адреса соседних сетей, с которыми данный маршрутизатор непосредственно связан;
- адреса соседних маршрутизаторов, с которыми связан маршрутизатор;
- расстояние до соседних маршрутизаторов принимается равным 0 или 1 в зависимости от реализации.

Каждые 30 секунд маршрутизатор передает широковещательный RIP-пакет, содержащий пары (V, D), где V – адрес доступной сети, называемый **вектором**, а D – расстояние до этой сети, называемое **длиной вектора**.

В метрике RIP длина вектора измеряется *числом транзитных маршрутизаторов* (в хопх) между данным маршрутизатором и соответствующей сетью. На основании полученных таблиц "вектор-длина"

маршрутизатор вносит дополнения и изменения в свою маршрутную таблицу, определяя пути минимальной длины во все доступные сети.

Поскольку каналы связи могут иметь разные пропускные способности, в некоторых реализациях RIP *длина вектора умножается на весовой коэффициент, зависящий от скорости передачи данных по КС.*

Основное *достоинство* RIP и других протоколов типа DVA – *простота реализации.*

Недостатки RIP:

1) *медленная стабилизация* оптимальных маршрутов;
2) *большая загрузка сети* передаваемыми таблицами "вектор-длина", обусловленная двумя основными факторами:

- периодичностью передачи широковещательных пакетов, содержащих таблицы "вектор-длина" – пакеты передаются даже в том случае, если в сети не было никаких изменений;
- большим объёмом этих таблиц, который пропорционален числу подсетей, входящих в сеть.

Протоколы типа DVA целесообразно применять в небольших и относительно устойчивых сетях. В больших сетях периодически посылаемые широковещательные пакеты приводят к перегрузке сети и уменьшению пропускной способности.

LSA – алгоритмы обмена информацией о состоянии каналов, называемые также *алгоритмами предпочтения кратчайшего пути SPF* (Shortest Path First), основаны на динамическом построении маршрутизаторами карты топологии сети за счет сбора информации обо всех объединяющих их каналах связи.

Для этого маршрутизатор периодически тестирует состояние каналов с соседними маршрутизаторами, пометая каждый канал как "активный" или "неактивный". На практике для уменьшения слишком частой смены этих двух состояний применяется следующее правило: *«канал считается "активным" до тех пор, пока значительный процент тестов не даст отрицательного результата, и "неактивным" – пока значительный процент тестов не даст положительного результата».*

При изменении состояния своих каналов маршрутизатор немедленно распространяет соответствующую информацию по сети всем остальным маршрутизаторам, которые, получив сообщения, обновляют свои карты сети и заново вычисляют кратчайшие пути во все точки назначения.

Достоинства алгоритмов LSA:

1) *гарантированная и более быстрая стабилизация* маршрутов, чем в алгоритмах DVA;
2) *простота отладки и меньший объем* передаваемой информации, не зависящий от общего числа подсетей в сети.

Протоколы типа LSA используются в больших или быстро растущих сетях. К ним относятся протоколы Open Shortest Path First (OSPF) и Intermediate System to Intermediate System (IS-IS).

Протоколы типа LSA, в отличие от DVA, посылают информацию о маршрутах только *при изменении сетевых соединений*.

Другое отличие заключается в *возможности выбора канала передачи* из нескольких возможных с учетом одного из параметров маршрутизации, задаваемого пользователем:

- задержки или скорости передачи данных;
- пропускной способности или производительности;
- надежности.

Слайд 42

Протокол OSPF

Самой распространенной реализацией алгоритма LSA является протокол OSPF – открытый стандарт, разработанный для применения в маршрутизаторах сети Интернет и широко используемый в настоящее время в других сетях (NetWare, SNA, XNS, DECNet). Протокол OSPF относится к протоколам внутренней маршрутизации – информация о доступных маршрутах между маршрутизаторами распространяется в пределах одной автономной системы.

Для построения таблицы маршрутизации протокол OSPF использует **алгоритм SPF** (Shortest Path First - «выбор кратчайшего пути»), который строит кратчайший путь к каждой сети в виде *дерева*, корнем которого является сам маршрутизатор, а ветвями – пути к доступным сетям (маршрутизаторам). Процесс построения дерева реализуется с использованием **алгоритма Дейкстры**.

Для нахождения кратчайших путей от некоторой вершины (маршрутизатора) до остальных вершин (маршрутизаторов) сопоставляется **метка**, определяющая минимальное расстояние от этой вершины, в виде метрики – количественной меры расстояния, учитывающей не только длину маршрута в сети, но и, например, пропускную способность каналов.

Алгоритм реализуется пошагово, причем на каждом шаге предпринимается попытка уменьшить метку одной вершины.

На первом шаге метка исходной вершины 1 полагается равной 0, а метки остальных вершин – бесконечности, означающей, что расстояния от вершины 1 до других вершин пока неизвестны. Все вершины графа помечаются как непройденные.

На каждом шаге из непройденных вершин выбирается вершина, имеющая минимальную метку.

На следующем шаге рассматриваются всевозможные маршруты, в которых вершина 1 является предпоследним пунктом. Вершины, в которые ведут рёбра из вершины 1, являются соседями этой вершины. Для каждого соседа

вершины 1, кроме отмеченных как непройденные, определяется новая длина пути, равная сумме значений текущей метки и длины ребра, соединяющего вершину 1 с этим соседом. Если полученное значение длины меньше значения метки соседа, значение метки заменяется полученным новым значением длины. После просмотра всех соседей, вершина отмечается как пройденная. Этот шаг алгоритма повторяется для остальных вершин до тех пор, пока не будут пройдены все вершины.

На основе построенного дерева для узла (маршрутизатора) 1 формируется таблица маршрутизации, содержащая для каждого адреса назначения основной и альтернативный пути с соответствующими метриками.

В процессе реализации протокола OSPF между маршрутизаторами осуществляется обмен служебными сообщениями в виде OSPF-пакетов, инкапсулируемых в IP-пакеты:

- **Hello-пакет** – установление и поддержание связей с соседними маршрутизаторами;
- **Database Description** – содержание базы данных состояния канала;
- **Link State Request** – запрос части базы данных соседнего маршрутизатора;
- **Link State Update** – рассылка объявлений о состоянии канала;
- **Link State Acknowledgment** - подтверждение получения пакета Link State Update.

Слайд 43

Протокол OSPF

Протокол OSPF поддерживает следующие типы сетей:

- широковещательные сети с множественным доступом (локальные сети *Ethernet*, *Token Ring*);
- точка-точка (коммутируемый доступ);
- нешироковещательные сети с множественным доступом (*Frame relay*).

В отличие от протокола RIP, использующего обычно в качестве метрики маршрута количество промежуточных маршрутизаторов (хопов), в качестве метрики протокола OSPF служит стоимость (cost) маршрута, учитывающая пропускную способность канала связи C_{KC} :

$$M_{OSPF} = K / C_{KC},$$

где K – максимальная пропускная способность КС на пути передачи данных (по умолчанию $K=10^8$), причем M_{OSPF} – целое число и $M_{OSPFmin} = 1$.

В случае составного соединения, включающего на маршруте n каналов связи с разными пропускными способностями, стоимость маршрута

$$M_{OSPF} = \sum_{i=1}^n M_i,$$

где M_i - стоимость i -го КС ($i = 1, \dots, n$).

Пусть между двумя оконечными узлами имеется два возможных маршрута передачи пакетов: первый маршрут содержит 2 канала с пропускными способностями $C_1=100$ Мбит/с и $C_2=10$ Мбит/с, а второй маршрут содержит 3 канала с пропускными способностями $C_3=100$ Мбит/с, $C_4=1$ Гбит/с и $C_5=100$ Мбит/с.

Рассчитаем метрики (стоимости) этих маршрутов с учетом того, что $K=10^9$: $M_1=10+100=110$ и $M_2=10+1+10=21$. Таким образом, несмотря на то что количество хопов на первом маршруте меньше, чем на втором, предпочтение в протоколе OSPF будет отдано второму маршруту, поскольку $M_2 < M_1$.

Алгоритм реализации протокола OSPF:

1. По умолчанию каждые 10 с – для широковещательных сетей и сетей точка-точка или 30 с – для нешироковещательных сетей с множественным доступом посылается широковещательное сообщение – **hello-пакет**.

2. Ожидание ответа.

3. Если в течение времени ожидания (5 секунд) поступает ответ, считается, что «канал активный».

4. Если же в течение 5 секунд ответ не поступит, то посылается повторный запрос.

5. Если нет ответа в течение 4-х кратного интервала посылки (40 с или 120 с) – канал считается «неактивным» и маршрутизатор распространяет соответствующую информацию всем остальным маршрутизаторам.

Обладая всеми преимуществами алгоритмов LSA, протокол OSPF обеспечивает следующие дополнительные возможности.

1. *Маршрутизация* пакетов в соответствии с заказанным типом обслуживания. Сетевой администратор может присваивать межсетевым каналам различные значения "стоимости", основываясь на их пропускной способности, задержках или эксплуатационных расходах. Маршрутизатор выбирает путь следования пакета в результате анализа не только адреса получателя, но и поля "тип обслуживания" в заголовке.

2. *Равномерное распределение нагрузки* между альтернативными путями одинаковой стоимости (в отличие от протокола RIP, вычисляющего только один путь в каждую точку назначения).

3. *Маршрутизация* пакетов в соответствии с классом обслуживания. Сетевой администратор может создать несколько очередей с различными приоритетами. Пакет помещается в очередь на отправку по результатам анализа типа протокола. Для пакетов, чувствительных к временным задержкам, выделяется очередь с более высоким приоритетом.

4. *Аутентификация маршрутов*. Отсутствие этой возможности, например в протоколе RIP, может привести к перехвату пакетов злоумышленником, который будет передавать таблицы "вектор-длина" с указанной малой длиной путей от своего ПК во все подсети.

5. Создание виртуального канала между маршрутизаторами, соединенными не напрямую, а через некоторую транзитную сеть.

Основные преимущества OSPF по сравнению с RIP:

- более высокая скорость сходимости;
- поддержка сетевых масок;
- более эффективное использование пропускной способности сети за счет построения дерева кратчайших путей

Слайд 44

3.7. Протокол межсетевых управляющих сообщений ICMP

ICMP (Internet Control Message Protocol) – протокол межсетевых управляющих сообщений в стеке TCP/IP, предназначенный для выявления и обработки нештатных событий (например, потеря пакета, превышение TTL, отсутствие адресата и т.п.), определения типа ошибки, формирования сообщения о ней и передаче этого сообщения приложению, сформировавшему пакет.

Протокол ICMP реализуется на сетевом уровне OSI-модели и частично выполняет некоторые функции транспортного уровня.

К функциям протокола ICMP относятся:

- обмен тестовыми сообщениями для выяснения наличия и активности узлов сети;
- анализ достижимости узла-получателя и сброс пакетов, направляемых к недостижимым узлам;
- изменение маршрутов;
- уничтожение пакетов с истекшим временем жизни;
- синхронизация времени в узлах сети;
- управление потоком путем регулирования частоты посылки пакетов узлами-источниками.

Одной из наиболее интересных среди перечисленных функций является изменение маршрутов: если некоторый маршрутизатор определяет, что хост использует неоптимальный путь для доставки пакета, он при помощи протокола ICMP может скорректировать маршрутную таблицу хоста. Это один из механизмов автоматической оптимизации и адаптации сетей TCP/IP к изменениям топологии.

Протокол IP не гарантирует доставку IP-пакета, не отправляет сообщения об ошибке и не пытается передать этот пакет заново. Эти функции возложены на **протокол ICMP**, который формирует соответствующее *сообщение*:

- при возникновении ошибки или нестандартной ситуации;
- узлу-отправителю в том случае, если запрашиваемая услуга или узел-получатель не достижимы;
- если необходима фрагментация пакета, у которого флаг DF = 1 (Don't fragment);

- когда TTL (Time to live) некоторого IP-пакета становится равным нулю (при этом пакет уничтожается, и узлу-отправителю направляется ICMP-сообщение о превышении TTL).

Так как протокол IP предоставляет сервис без гарантии доставки, то ICMP-сообщение об ошибках не обязаны обрабатываться ни протоколом ICMP, ни протоколом IP, ни каким-либо другим протоколом.

Формат ICMP-сообщения

Рассмотрим формат ICMP-сообщения.

Заголовок длиной 8 байт содержит следующие поля:

- тип (1 байт);
- код (1 байт);
- контрольная сумма (2 байта);
- поле служебной информации, зависящее от типа и кода.

Тип и **код** определяют содержание сообщения: какая произошла ошибка, причина ошибки, диагностическое действие.

Контрольная сумма используется для проверки правильности доставки ICMP-пакета.

Следующие 4 байта – **служебная информация**, которая зависит от типа и кода сообщения.

В **поле данных** ICMP, как правило, включается фрагмент пакета, при передаче которого произошла ошибка.

ICMP-сообщения инкапсулируются в IP пакеты. В зависимости от версии протокола IP различают и версии ICMP: протокол ICMPv4 для IPv4 и протокол ICMPv6 для IPv6.

Протокол ICMP, как и протокол UDP, является «ненадежным», т.е. не гарантирует доставку пакетов и отсутствие ошибок. В отличие от UDP, где обеспечение надёжности возложено на программные средства прикладного уровня, протокол ICMP обычно не нуждается в реализации надёжной доставки, т.к. он в отличие от TCP и UDP, как правило, не используется для передачи и приёма данных между конечным оборудованием и непосредственно в приложениях пользователей сети. Исключение из правил составляют две утилиты: Ping и Tracert (Traceroute), использующие протокол ICMP для передачи сообщений.

Типы ICMP-сообщений, информирующих о событиях, происходящих в сети, представлены ниже в таблице.

Как видно из таблицы, **тип 0 и тип 8** – это соответственно эхо-ответ и эхо-запрос, которые используются утилитой Ping для проверки доступности компьютера в сети (возможности доставки IP-пакетов).

ICMP-сообщения типа 3 генерируются маршрутизатором при отсутствии маршрута к адресату, что свидетельствует о недостижимости узла назначения.

ICMP-сообщения типа 5 используются маршрутизаторами для обновления записей в таблице маршрутизации узла-отправителя при обнаружении более «быстрого» маршрута.

ICMP-сообщения типа 9 периодически рассылают в сети, чтобы компьютеры могли узнать о существовании конкретных маршрутизаторов.

ICMP-сообщения типа 10 представляют собой запрос от компьютера, который подключился к сети и, не ожидая периодического сообщения от маршрутизаторов, желает получить информацию о них, которая содержится в ответном сообщении от маршрутизатора.

Тип	Назначение сообщения
0	Эхо-ответ
3	Узел назначения недостижим
5	Перенаправления маршрута
8	Эхо-запрос
9	Сообщение о маршрутизаторе
10	Запрос сообщения о маршрутизаторе
11	Истечение времени жизни пакета
12	Проблемы с параметрами
13	Запрос отметки времени
14	Ответ отметки времени

ICMP-сообщение типа 11 используется при достижении времени жизни IP-пакета значения TTL=0, что может быть обусловлено появлением в сети петли. Эти же ICMP-сообщения используются утилитой Traceroute (Tracert), позволяющей проследить путь следования IP-пакетов.

ICMP-сообщения (тип 12) генерируются при обнаружении ошибок в заголовке IP-пакета, что не позволяет маршрутизатору отправить такой пакет. Исключение составляют IP-пакеты с ICMP-сообщениями, чтобы не привести к лавинообразному потоку новых ICMP-сообщений об ошибках в ICMP-сообщениях.

Код ICMP-сообщения информирует о причине появления этого сообщения. В таблице на слайде представлено несколько кодов (причин) для типа сообщения 3 – узел назначения недостижим, а также типов 11 и 12.

Специфические особенности протокола ICMP приведены на слайде.

ICMP-сообщения инкапсулируются в IP пакеты. ICMP является неотъемлемой частью IP, но при этом не делает протокол IP средством надёжной доставки сообщений. Для этих целей существует протокол TCP.

Слайд 45

Применение ICMP

Протокол межсетевых управляющих сообщений ICMP используется утилитами ping и tracert (traceroute).

Утилита командной строки **ping** реализует проверку доступности компьютера или сервера в сети по заданному DNS- или IP-адресу.

По умолчанию посылается четыре ICMP-сообщения с типом 8 и кодом 0, называемые «эхо-запросом», длиной 32 или 64 байт, на которые возвращаются четыре эхо-ответа с типом 0 и кодом 0, содержащие диагностическую информацию о времени (минимальном, максимальном и среднем) приема-передачи в миллисекундах и TTL, характеризующем количество маршрутизаторов на пути передачи.

Утилита **tracert** в Windows (tracert в MAC и Linux) позволяет определить маршрут от отправителя к получателю (например, от клиента к серверу) и выявить проблемы, связанные с доступностью сервера.

По умолчанию последовательно к каждому маршрутизатору на пути передачи данных всякий раз посылается по 3 ICMP-пакета с одним и тем же значением TTL и в течение 4 секунд ожидается ответ. Первые 3 пакета имеют TTL=1. Затем значение TTL последовательно увеличивается на 1, что позволяет определить время передачи данных для первого, второго и т.д. маршрутизаторов на пути передачи данных. В ответном пакете от каждого маршрутизатора содержится информация о времени достижения маршрутизатора и его IP-адрес.

В некоторых случаях вместо времени достижения и IP-адреса от маршрутизатора приходит сообщение о превышении интервала ожидания для запросов, которое составляет 4 секунды. Причинами этого могут быть следующие:

- маршрутизатор не принимает и не обрабатывает ICMP-пакеты;
- маршрутизатор не связан со следующим узлом;
- не указан маршрут к данному узлу на предыдущем маршрутизаторе.

Слайд 46

3.8. Коммуникационный протокол IPv6

Проблемы, с которыми в начале 90-х годов столкнулись разработчики и пользователи Интернета, базирующегося на протоколах TCP/IP, привели к осознанию необходимости разработки новой версии протокола IP – ***протокола IPv6***, который должен обеспечить достижение следующих **целей**:

- создание *масштабируемой системы адресации*, обеспечивающей поддержку миллиардов хостов даже при неэффективном использовании адресного пространства;
- *уменьшение размера таблиц маршрутизации* и упрощение протокола для ускорения обработки пакетов маршрутизаторами;
- предоставление *гарантий качества транспортных услуг* при передаче неоднородного трафика, в частности, при передаче данных реального времени;
- более надёжное обеспечение безопасности – аутентификации и конфиденциальности;
- возможность развития протокола в будущем;
- возможность сосуществования старого (IPv4) и нового (IPv6) протоколов.

Основными особенностями протокола IPv6 являются следующие.

1. Длина IP-адреса увеличена до 16 байт (128 бит), что предоставляет пользователям практически неограниченное адресное пространство.
2. Упрощена структура заголовка, содержащего всего 7 полей (вместо 13 в протоколе IPv4), что позволяет маршрутизаторам быстрее обрабатывать пакеты, то есть повышает их производительность. Уменьшен размер таблиц маршрутизации. Фрагментация, при необходимости, реализуется в конечных узлах, а маршрутизация может быть задана от источника, что уменьшает время доставки пакетов и увеличивает производительность сети.
3. Улучшена поддержка необязательных параметров, так как в новом заголовке требуемые прежде поля стали необязательными, а изменённый способ представления необязательных параметров ускоряет обработку пакетов в маршрутизаторах за счёт пропуска не относящихся к ним параметров.
4. Улучшена система безопасности за счет обязательного шифрования с использованием защищенного протокола IPSec (Security Internet Protocol).
5. Предусмотрена возможность расширения типов (классов) предоставляемых услуг, которые могут появиться в результате ожидаемого роста мультимедийного трафика.

Слайд 47

Коммуникационный протокол IPv6: адресация

Необходимость расширения адресного пространства в сетях TCP/IP была одной из основных целей перехода на новую версию протокола IP. Для этого длина IP-адреса была увеличена до **16 байт** или **128 бит**, что предоставляет пользователям практически *бесконечное адресное пространство* – более чем 10^{38} адресов.

В протоколе IPv6 вместо двухуровневой иерархии адресов, как в IPv4, используется четырёхуровневая:

- 3 уровня используются для идентификации сетей;
- 1 уровень используется для идентификации узла сети.

В протоколе IPv6 предусмотрено 3 типа IP-адресов:

- **индивидуальные адреса** (unicast), определяющие идентификаторы интерфейсов оконечных узлов (хостов) или маршрутизаторов;
- **групповой адрес** (multicast), аналогичный групповому адресу IPv4, идентифицирует группу интерфейсов, относящихся, как правило, к разным узлам;
- **адрес произвольной рассылки** (anycast) – адрес, назначаемый только интерфейсам маршрутизатора и определяющий группу интерфейсов, к одному из которых доставляется пакет с таким адресом, как правило, «ближайшему» в соответствии с метрикой, используемой протоколами маршрутизации.

Индивидуальные IP-адреса могут быть трёх типов:

- **глобальный агрегируемый уникальный адрес** - основной тип индивидуального адреса, основанного на агрегировании для упрощения маршрутизации; уникальность адресов в Интернете обеспечивается за счет их централизованного распределения (IANA);

- **локальные адреса подсети (SubNet, SN)**, используемые только в пределах сети (подсети) организации (без обращения в IANA), поскольку такие адреса не маршрутизируются;

- **локальный адрес канала связи (Link Local, LL)**, который *не маршрутизируется*, назначается автоматически и действителен в пределах одного сегмента сети, одного коммутатора или нескольких связанных между собой коммутаторов.

Как и в версии IPv4, сохранены два специальных адреса:

- **адрес обратной петли**, играющий ту же роль, что и адрес 127.0.0.1 протокола IPv4, и имеющий вид: 0:0:0:0:0:0:0:1 или ::1 ;

- **неопределённый адрес**, состоящий из одних нулей (0:0:0:0:0:0:0:0 или ::) и являющийся аналогом адреса 0.0.0.0 протокола IPv4.

Рассмотрим структуру глобального агрегируемого уникального адреса.

Поле **FP** (Format Prefix – **префикс формата**) определяет формат адреса и для рассматриваемого типа имеет значение 001.

Следующие поля описывают три уровня идентификации сетей:

- **TLA** (Top-Level Aggregation – **агрегирование верхнего уровня**) предназначено для нумерации сетей самых крупных поставщиков услуг; небольшое количество разрядов (13 двоичных разрядов) позволяют ограничить количество таких сетей числом 8195 и, следовательно, ограничить размер таблиц маршрутизации и ускорить работу магистральных маршрутизаторов; следующие 8 разрядов за полем TLA зарезервированы на будущее для его расширения;

- **NLA** – (Next-Level Aggregation – **агрегирование следующего уровня**) предназначено для нумерации средних и мелких поставщиков услуг;

- **SLA** – (Site-Level Aggregation – **агрегирование местного уровня**) предназначено для нумерации подсетей, находящихся в распоряжении одного администратора, который может формировать адреса, состоящие из идентификатора подсети SLA и идентификатора интерфейса IdInt, без согласования с поставщиком услуг.

Поле **IdInt** – идентификатор интерфейса является аналогом номера узла в протоколе IPv4, но в отличие от него содержит физический (локальный) адрес интерфейса (например, MAC-адрес или адрес X.25), а не произвольно назначенный номер узла. В этом случае *отпадает необходимость в протоколе ARP* и в ручном конфигурировании конечных узлов. В IPv6 функциональность ARP обеспечивает **протокол обнаружения соседей NDP** (*Neighbor Discovery Protocol*). Кроме того, *теряет смысл использование масок для разделения сетей на подсети*, в то время как объединение сетей приобретает особое значение.

Для записи 16-байтовых адресов используется *шестнадцатеричная форма* (вместо десятичной формы в протоколе IPv4), причём каждые 4 шестнадцатеричные цифры отделяются друг от друга *двоеточием*:

AB25:164:0:E12B:6:0:C2C4:1234

Слайд 48

Адресация: групповой адрес

Групповой адрес идентифицирует группу интерфейсов, возможно, с разным расположением, для получения одного и того же контента. Групповые адреса имеют префикс ff::/8. Пакет с групповым адресом доставляется всем членам группы.

Базовая структура группового адреса IPv6 включает 4 поля:

- **FP** – формат префикса длиной 8 бит, значения которых равны 1;
- **Flag** – определяет тип адреса и может принимать следующие значения:
 - **0000** – фиксированный адрес, назначенный организацией IANA (Internet Assigned Numbers Authority) по распределению имён, номеров и адресов в сети Интернет;
 - **0001** – временный (локально выделенный) адрес;
- **Scope** – определяет диапазон адресов группы.

Слайд 49

Адресация: примеры адресов IPv6

На слайде представлены примеры адресов IPv6, правила отображения адресов в сокращенном виде.

Как было показано выше, для записи адресов IPv6 используется *шестнадцатеричная форма*, причём каждые 4 шестнадцатеричные цифры отделяются друг от друга *двоеточием*:

AB25:164:0:E12B:6:0:C2C4:1234

BDA5::3217:19:0:F084 .

Как видно из представленных на слайде примеров, при записи адреса допускается ряд упрощений:

- вместо 4-х нулей записывается только один ноль: **0** вместо 0000;
- можно опускать незначащие нули в начале каждого четырёхсимвольного поля адреса: **164** вместо 0164 или **6** вместо 0006;
- если в адресе имеется длинная последовательность нулей, то запись можно сократить, заменив в ней все нули двоеточием, причём двоеточие может употребляться только один раз:

CF18: 35::67:5 , что соответствует адресу **CF18: 35:0:0:0:0:67:5** ;

- для сетей, использующих обе версии (IPv4 и IPv6) протокола разрешается использовать традиционную десятичную запись IPv4 в 4-х младших байтах, например: **::BAC2:192.85.1.6** .

Преимущества адресации IPv6:

1. Ускорение маршрутизации: всего 8192 сети верхнего уровня (TLA).
2. Возможность указать непосредственно физический адрес устройства (MAC- адрес).
3. Не нужен ARP-протокол и ручное конфигурирование конечных узлов (протокол обнаружения соседей NDP - Neighbor Discovery Protocol).
4. Не нужно маскирование адресов.

Слайд 50

Структура пакета и формат заголовка IPv6

Структура пакета IPv6 существенно отличается от пакета IPv4. Это проявляется, прежде всего, в возможности наличия нескольких заголовков – кроме основного заголовка, который всегда присутствует, пакет может иметь несколько дополнительных заголовков, которые могут содержать информацию, необходимую для качественной передачи пакета.

В качестве дополнительных заголовков могут использоваться следующие:

- **заголовок маршрутизации**, содержащий полный маршрут при маршрутизации от источника;
- **заголовок фрагментации**, содержащий информацию о фрагментации исходного IP-пакета;
- **заголовок аутентификации**, содержащий информацию, необходимую для аутентификации конечных узлов и обеспечения целостности содержимого IP-пакетов;
- **заголовок системы безопасности**, содержащий информацию, необходимую для обеспечения конфиденциальности передаваемых данных путём шифрования пакетов;
- **специальные параметры**, необходимые для обработки пакетов в процессе передачи по сети;
- **параметры получателя**, содержащие дополнительную информацию для узла назначения.

Такая структура пакета IPv6 обеспечивает следующие **преимущества** по сравнению с пакетом IPv4:

- *меньше нагрузка на маршрутизаторы*, поскольку все дополнительные заголовки обрабатываются только в конечных узлах;
- *большая функциональность и открытость* для внедрения новых механизмов протокола IP за счёт использования большого количества дополнительных параметров.

- *большая функциональность и открытость* для внедрения новых механизмов протокола IP за счёт использования большого количества дополнительных параметров.

Основной заголовок IPv6 имеет фиксированную длину 40 байт и содержит следующие поля.

Поле «Версия» (4 бита) содержит число 6 для пакета IPv6.

Поле «Класс трафика» (8 бит) используется для того, чтобы различать пакеты с разными требованиями к доставке в реальном времени, и включает 6 битов под приоритет пакета и 2 бита – признак явного уведомления о перегрузке ECN.

Поле «Метка потока» предназначено для установления между отправителем и получателем псевдосоединения с определёнными свойствами и требованиями. Маршрутизаторы, в зависимости от метки потока в заголовке прибывшего пакета, определяют, какого рода особая обработка требуется пакету. С помощью этого поля протокол пытается объединить достоинства дейтаграммного способа передачи пакетов и способа «виртуальный канал».

Поле «Длина полезной нагрузки» указывает, сколько байт содержится в пакете *без учета основного заголовка*, длиной 40 байт. Аналогичное поле «Полная длина» в IPv4 определяло *всю длину пакета с учётом заголовка*.

Поле «Следующий заголовок» указывает, какой из дополнительных заголовков следует за основным. Все дополнительные заголовки содержат такие же поля, которые указывают на последующие заголовки. В последнем заголовке в этом поле указывается протокол, которому следует передать содержимое пакета: ICMP (1), TCP (6), UDP (17), OSPF (89), RIP (520),

Поле «Максимальное число транзитных участков (ТУ)» определяет время жизни пакета. Значение поля, устанавливаемое узлом-отправителем, уменьшается на единицу на каждом транзитном участке.

Далее следуют 16-байтные IP-адреса отправителя и получателя.

Сравнение заголовка IPv6 с заголовком IPv4 показывает, что:

- поле «Длина заголовка» исчезло, так как основной заголовок IPv6 имеет фиксированную длину, но появилось поле «Длина полезной нагрузки», характеризующая размер дополнительных заголовков;

- поле «Протокол» отсутствует, поскольку поле «Следующий заголовок» указывает, что следует за последним заголовком (ICMP-пакет, TCP-сегмент, UDP-дейтаграмма и т.д.);

- удалены поля, относящиеся к фрагментации, так как все узлы, поддерживающие протокол IPv6, должны динамически определять нужный размер дейтаграммы, что делает фрагментацию маловероятной;

- минимальный размер пакета, который должен передаваться в сетях IPv6 без фрагментации, увеличен с 576 до 1280 байт;

- поле «Контрольная сумма» удалено, так как её подсчёт занимает много времени, что существенно снижает производительность узлов; к тому же всё шире используются надёжные волоконно-оптические линии связи.

Таким образом, протокол IPv6 является *простым, быстрым и гибким* протоколом сетевого уровня с огромным адресным пространством.

Слайд 51

Сравнение заголовков IPv4 и IPv6

На слайде для сравнения ещё раз показаны заголовки пакетов протоколов IPv4 и IPv6.

Слайд 52

Совместимость протоколов IPv6 и IPv4

Поскольку протокол IPv6 невозможно одномоментно внедрить во всех подсетях Интернета, переход на IPv6 происходит постепенно, что привело к необходимости обеспечить взаимодействие обеих версий протокола IP: IPv6 и IPv4. В связи с этим появилась необходимость разработки способов межсетевого взаимодействия сетей, функционирующих на основе разных версий протокола IP, а именно: IPv4 и IPv6. Можно выделить три способа взаимодействия:

- трансляция протоколов, когда в шлюзе (маршрутизаторе), связанном с сетями IPv4 и IPv6, реализуется преобразование адресов;
- мультиплексирование стеков протоколов IPv4/ IPv6 в конечных узлах и во всех маршрутизаторах;
- инкапсуляция (туннелирование).

Трансляция протоколов реализована следующим образом.

Для того чтобы узлы, поддерживающие протокол IPv6, могли передавать пакеты через сеть IPv4, разработан специальный адрес: **IPv4-совместимый IPv6-адрес**, который переносит адрес IPv4 в младших 4-х байтах адреса IPv6, а в 12 старших байтах содержатся нули.

Для передачи пакетов IPv4 через подсети, работающие по протоколу IPv6 предназначен **IPv4-отображённый IPv6-адрес**, содержащий в первых десяти байтах нули, а в двух последующих байтах – единицы, которые показывают, что данный узел поддерживает только протокол IPv4.

Мультиплексирование стеков протоколов IPv4/IPv6 в конечных узлах и во всех маршрутизаторах предполагает наличие в каждом из узлов обеих версий протокола IP.

Инкапсуляция или туннелирование реализуется только при передаче пакетов IPv4 через сети IPv6.

Слайд 53

3.9. Протоколы канального уровня для выделенных линий

Протоколы канального уровня для выделенных линий должны:

- обеспечивать надежную передачу;

- предоставлять возможность управления потоком кадров для предотвращения переполнения соседних узлов.

К протоколам канального уровня относятся следующие:

- SLIP;
- протоколы семейства HDLC;
- PPP (PPTP).

Протокол SLIP (Serial Line IP) – первый стандарт для протоколов TCP/IP, который может использоваться как для коммутируемых, так и для выделенных каналов ввиду простоты.

SLIP поддерживается только протоколом сетевого уровня IP. Основная и единственная *функция* протокола SLIP – распознавание начала и конца IP-пакета в потоке бит. Для этого в качестве границ IP-пакета используется специальный символ END (шестнадцатеричный код – C0₁₆). Если в IP-пакете встречается код C0₁₆, то используется процедура **байт-стаффинга**, заключающаяся в следующем. Код C0₁₆ заменяется на коды DB₁₆ и DC₁₆, а код DB₁₆ заменяется на DB₁₆ и DD₁₆.

К недостаткам протокола SLIP относятся:

- отсутствие возможности обмениваться адресной информацией;
- использование только IP-пакетов в качестве содержимого SLIP-кадра;
- отсутствие процедур обнаружения и коррекции ошибок;
- высокая избыточность из-за передачи полных заголовков верхних уровней и байт-стаффинга.

Слайд 54

Протокол HDLC (High-level Data Link Control Procedure)

HDLC – высокоуровневый протокол управления каналом – стандарт ISO для выделенных линий. Представляет собой *семейство протоколов LAP* (Link Access Protocol), включающее следующие протоколы:

- **LAP-B** – для сетей X.25 (B – Balanced);
- **LAP-D** – для сетей ISDN (D – D-channel);
- **LAP-M** – для модемов (M – Modem);
- **LAP-F** – для сетей Frame Relay (F – Frame Relay).

Протокол рассматривает сеть как множество взаимодействующих узлов (станций/терминалов), в качестве которых могут выступать хосты, маршрутизаторы, коммутаторы. Станции могут быть трех типов:

- первичная (ведущая) (PT – *Primary Terminal*);
- вторичная (ведомая) (ST – *Secondary Terminal*);
- комбинированная (СТ – *Combined Terminal*).

Первичные станции реализуют функции:

- управления каналом;
- обнаружения ошибок;

- восстановления работоспособности.

Станции обеспечивают следующие режимы логического соединения.

1. **Режим нормального ответа** (Normal Response Mode, NRM) – требует явного разрешения на передачу от первичной станции; круговой опрос вторичных станций в соединениях точка-многоточка.
2. **Режим асинхронного ответа** (Asynchronous Response Mode, ARM) – вторичная станция может сама инициировать передачу в соединениях типа кольцо и многоточечных по типу маркера (token).
3. **Асинхронный сбалансированный режим** (Asynchronous Balanced Mode, ABM) используется комбинированными станциями в *дуплексном режиме* (оба устройства равноправны и обмениваются командами и ответами).

Возможны два типа соединений:

- синхронные, использующие механизм битстаффинга;
- асинхронные, использующие механизм байтстаффинга.

Синхронность соединения достигается за счет межкадрового временного заполнения восьмибитовыми последовательностями **0111110 0111110 0111110**, что обеспечивает битовую синхронизацию взаимодействующих станций.

Восьмибитовая последовательность **0111110** используется в качестве разделителя кадров при их передаче.

Слайд 55

Формат кадра HDLC

HDLC относится к бит-ориентированным протоколам и использует кадр, формат которого показан на слайде.

В качестве обрамления кадра, служащих границами между передаваемыми кадрами, используется специальная последовательность из 8 бит: **01111110**, называемая *флагом*. Благодаря наличию флагов нет необходимости указывать длину кадра. Для того, чтобы отличать последовательность бит 01111110, находящуюся в поле данных от флага применяется процедура *бит-стаффинга*.

Поле **Адрес** имеет длину 1 или 2 байта и при наличии нескольких узлов-приёмников используется для идентификации конкретного узла, а в двухточечном соединении – для того, чтобы отличить команды от ответов, а также для указания направления передачи кадра по интерфейсу «пользователь – сеть».

Поле **Данные** может быть любой длины и содержать пакеты протоколов вышележащих уровней. Это поле может отсутствовать в управляющих кадрах и некоторых нумерованных кадрах.

Поле **КС** (контрольная сумма) содержит *остаток избыточной циклической суммы*, вычисленной с помощью полиномов типа CRC.

Поле **У** (*управление*) имеет длину 1 или 2 байта и содержит служебную информацию. Структура и содержимое этого поля зависят от типа передаваемого HDLC-кадра.

Существуют 3 типа HDLC-кадров, различающиеся содержимым поля **У** (*управление*):

- *информационные кадры* длиной 1 или 2 байта, предназначенные для передачи данных пользователя;
- *управляющие* или *супервизорные кадры* длиной 1 или 2 байта, предназначенные для передачи команд и ответов в процессе установленного логического соединения;
- *нenumерованные кадры* длиной 1 байт, предназначенные для установления и разрыва логического соединения, а также информирования об ошибках.

Тип кадра определяется первыми битами поля управления: 0 – информационный кадр; 10 – управляющий кадр; 11 – нenumерованный кадр.

Протокол HDLC для обеспечения надёжности передачи данных использует механизм скользящего окна, ширина которого составляет:

- 7 кадров при длине поля управления в 1 байт;
- 127 кадров при длине поля управления в 2 байта.

Для реализации механизма скользящего окна в **информационном кадре** предусмотрено 2 поля:

- поле **N(S)**, содержащее порядковый номер передаваемого кадра;
- поле **N(R)**, содержащее номер очередного ожидаемого кадра.

Наличие этих двух полей связано с реализацией дуплексной передачи данных, а их длина определяет ширину окна в 7 (при длине 3 бита) или 127 (при длине 7 бит) кадров.

Бит **P/F** (Poll/Final – Опрос/Финал) используется для указания промежуточного (P) или последнего передаваемого (F) кадра. В некоторых случаях этот бит может использоваться для указания другому узлу о необходимости передать управляющий кадр, не ожидая попутного потока данных.

Управляющие кадры могут быть 4-х типов, которые различаются значением поля **Type**:

- **Type=0** – *подтверждение* (RESEIVE READY – к приёму готов) – передаёт в поле **N(R)** номер следующего ожидаемого кадра и используется при отсутствии попутного потока данных для передачи подтверждения;
- **Type=1** – *отрицательное подтверждение* (REJECT – отказ) – передаёт в поле **N(R)** номер неверно полученного кадра, начиная с которого узел-отправитель должен повторить передачу кадров;
- **Type=2** – *отказ* (RESEIVE NOT READY – к приёму не готов) – означает, что в узле-получателе возникли проблемы, не позволяющие принимать кадры (например, переполнена буферная память) и, соответственно, узел-отправитель должен приостановить передачу кадров, при этом в поле **N(R)** указывается номер

кадра, начиная с которого узел-отправитель в дальнейшем (после устранения причины приостановки приёма кадров) должен будет повторить передачу кадров;

- **Type=3** – *выборочное подтверждение* (SELECTIVE REJECT – выборочный отказ) – передаёт в поле **N(R)** номер только того кадра, передачу которого узел-отправитель должен повторить.

Ненумерованные кадры применяются в основном для служебных целей, но могут переносить и данные, когда требуется ненадёжный не требующий соединения сервис. Поля **Type** и **Modifier** определяют типы и модификации команд, используемых двумя узлами на этапе установления соединения. Примерами таких команд могут служить:

- запрос на установление соединения с использованием двухбайтовых полей управления для информационных и управляющих кадров: SABME (Set Asynchronous Balanced Mode Extended – установить асинхронный сбалансированный расширенный режим);
- подтверждение установления или разрыва соединения: UA (Unnumbered Acknowledgment – ненумерованная положительная квитанция);
- запрос на разрыв соединения: REST (Resetting connection – сброс соединения).

Одна из основных функций протоколов семейства HDLC – восстановление искаженных и потерянных кадров (уменьшение вероятности искажения бита – BER с 10^{-4} до 10^{-9}). Для современных каналов высокого качества, обеспечивающих значение $BER=10^{-9}$ и выше, использование протоколов семейства HDLC на уровне маршрутизатора или коммутатора становится нецелесообразным.

Слайд 56

Протокол PPP (Point-to-Point Protocol)

PPP (Point-to-Point Protocol) – протокол двухточечного соединения, заменивший протокол SLIP и построенный на основе формата кадров протоколов семейства HDLC с дополнением собственных полей.

Протокол PPP является *байт-ориентированным* протоколом канального уровня и так же, как протокол HDLC, представляет собой семейство протоколов, включающее в том числе:

- **LCP** (Link Control Protocol) – *протокол управления соединением* (установка, поддержка и завершение соединения);
- **NCP** (Network Control Protocol) – *протокол управления сетью* для определения настроек сетевого уровня (сетевой адрес и пр.);
- **MLPPP** (Multi Link PPP) – *многоканальный протокол PPP* формирует несколько физических каналов для одного логического соединения;

- **PAP** (Password Authentication Protocol), **CHAP** (Challenge Handshake Authentication Protocol), **EAP** (Extensible Authentication Protocol) – *протоколы аутентификации*.

Протокол PPP основан на четырех принципах:

- *автоматическая настройка интерфейсов* за счет согласования параметров соединения (качество ЛС, размер кадров, тип протокола аутентификации) на основе переговоров (LCP);
- *многопротокольная поддержка* (IP, IPX, XNS, ...) за счет конфигурирования сетевого протокола (NCP);
- расширяемость протокола;
- независимость от глобальных служб.

Слайд 57

Формат кадра PPP для работы в нумерованном режиме

В отличие от бит-ориентированного протокола HDLC, протокол PPP является *байт-ориентированным*, что означает посимвольное заполнение кадра, то есть все кадры состоят из целого числа байтов. Полный формат кадра PPP для работы в нумерованном режиме показан на слайде

Характерными для PPP-кадра являются следующие особенности.

1. Если в поле **Данные** встречается байт 01111110, совпадающий с кодом флага, то используется процедура *байт-стаффинга*.

2. Поле **Адрес** всегда содержит значение 11111111, что означает, что все станции должны принимать этот кадр и позволяет избежать необходимости назначения адресов для передачи данных.

3. Поле **управления У** по умолчанию содержит значение 00000011, означающее *нумерованный кадр*.

4. Поле **Протокол** содержит код протокола вышележащего уровня, пакет которого вложен в поле данных. Длина этого поля по умолчанию составляет 2 байта, но путём переговоров длина может быть уменьшена до 1 байта.

5. Поле **Данные** может быть переменной длины, вплоть до некоторого установленного пользователями максимального значения, которое по умолчанию обычно составляет 1500 байт.

6. Поле **контрольной суммы КС** по умолчанию имеет длину 2 байта, которая в случае необходимости по договорённости может быть увеличена до 4-х байтов.

7. Установление соединения между двумя узлами сопровождается сложной переговорной процедурой принятия параметров соединения, таких как качество линии связи, размер передаваемых кадров, тип протокола аутентификации и т.д. Эта переговорная процедура реализуется протоколом управления линией связи LCP.

8. Протокол PPP реализует многопротокольную поддержку, обеспечивая внутри одного соединения передачу пакетов различных протоколов сетевого (IP, IPX, XNS и т.д.) и канального уровня локальных сетей.

Протокол PPTP (*Point-to-Point Tunneling Protocol*)

PPTP – туннельный протокол типа точка-точка, позволяющий установить защищённое соединение компьютера с сервером за счёт создания специального туннеля в незащищённой сети.

PPTP помещает (инкапсулирует) кадры PPP в IP-пакеты для передачи через сеть Интернет, как это показано на слайде. PPTP может также использоваться для организации туннеля между двумя локальными сетями. PPTP использует дополнительное TCP-соединение для обслуживания туннеля.